Domain-Oriented Masking Revisited: More Efficient AES Implementations with Arbitrary Protection Order

Feng Zhou[1,2,3], Hua Chen[2], Limin Fan[2] and Junhuai Yang[1,2]

[1] University of Chinese Academy of Sciences, Beijing, China
[2] TCA Laboratory, Institute of Software, Beijing, China
{zhoufeng2021,chenhua,fanlimin,yangjunhuai22}@iscas.ac.cn
[3] Zhongguancun Laboratory, Beijing, China

Abstract. Recent years have witnessed significant progress in composable masked AES designs based on Hardware Private Circuits (HPCs) under the Probe-Isolating Non-Interference (PINI) framework. However, these designs still suffer from substantial randomness requirements and area overhead at higher protection orders. In this work, we revisit Domain-Oriented Masking (DOM), originally proposed by Gross *et al.* in 2016, and leverage the DOM-*dep* and DOM-*indep* multipliers to construct efficient AES implementations based on the Strong Non-Interference (SNI) framework. Our contributions include: (1) a comprehensive security analysis of DOM-*dep* and DOM-*indep*, including their compositional security under the SNI framework; (2) more efficient masked AES implementations for arbitrary protection orders, reducing randomness and area overhead while maintaining latency comparable to state-of-the-art HPC3-based designs. Specifically, our masked AES implementations maintain a latency of 41 clock cycles by using the Hadzic's decomposition for $\mathbb{F}_{2^8}$ inverter. When $d \leq 4$, they save at least 13% in area (RNG included) and reduce latency by 19.6% compared to the smallest d-PINI round-based masked AES implementations provided by Cassiers *et al.* (The current version focuses on the core construction and its initial evaluation. Source code has been made publicly available to facilitate verification. Further performance optimizations and theoretical generalizations are underway and will appear in an upcoming revision.)

Keywords: SCA-resilient Hardware · Domain-Oriented Masking · Strong Non-Interference · Glitch-extended Probing Security

1 Introduction

Power analysis [KJJ99] has posed a significant threat to cryptographic implementations. In response, researchers have proposed various countermeasures to resist such attacks. Among them, masking [CJRR99, GP99] stands out as one of the most effective techniques. Although it was originally grounded in the theoretical Probing model [ISW03], practical physical defaults have been shown to violate the assumptions of this model [FGP+18]. In particular, glitches in hardware circuits have become a central concern. Nikova *et al.* [NRR06] introduced the first glitch-resistant masking scheme—Threshold Implementations (TIs). TIs use $td + 1$ shares to mask a cryptographic function of algebraic degree t, achieving d-th order security. Early TI constructions primarily targeted first-order security [BNN+12, KNP13], and were later extended to higher-order settings [BGN+14]. At Crypto 2015, Reparaz *et al.* [RBN+15] proposed Consolidated Masking Schemes (CMS), reducing the number of required shares to $d + 1$ for d-th order security. Subsequently, Gross *et al.*

[GMK16a] introduced Domain-Oriented Masking (DOM) at CCS 2016 as another $d+1$-share solution for hardware implementations. However, these techniques were developed without a formal model that accurately captures physical defaults. As a result, many were later found to have flaws under the robust probing model introduced by Faust *et al.* [FGP+18], as demonstrated by Moos *et al.* [MMSS19]. These flaws primarily stem from a lack of composition guarantees. Nonetheless, promising composable notions such as SNI (Strong Non-Interference) [BBD+16] had already emerged in the software domain by that time.

The construction of composable hardware circuits with arbitrary protection orders has been extensively studied in recent years. Among the most promising advances is the series of Hardware Private Circuits (HPC), including HPC1/2 [CGLS21], HPC3 [KM22], HPC3o [CGM+24], HPC3.1 [HB25], and HPC4 [CSV24]. While HPC1/2 are characterized by lower randomness consumption but higher latency, later variants such as HPC3 and beyond trade increased randomness for reduced latency. These gadgets have been adopted in practical implementations, particularly for AES [KMMS22, MCS22, KM22, CGM+24, HB25]. Early works [KMMS22, MCS22] based on HPC2 incurred substantial overhead in area, randomness, and latency—requiring six clock cycles per S-box evaluation. The latency of masked AES S-box implementations based on HPC3 [KM22, CGM+24] is reduced to four clock cycles. Notably, the design in [CGM+24] further reduces area and randomness overhead compared to the HPC2-based implementation [MCS22]. These improvements indicate that HPC3 variants have become the de facto paradigm for constructing compact and efficient masked circuits. Building on this direction, Hadžic *et al.* [HB25] proposed a $\mathbb{F}_{2^8}$ inverter with a multiplicative depth of three, enabling an S-box implementation with a latency of only three clock cycles. By leveraging randomness reuse and reducing the number of multiplications via factoring, their design achieves a randomness cost comparable to the 4-cycle S-box in [CGM+24], while further reducing both area footprint and latency.

In this paper, we adopt the (Strong) Non-Interference framework to construct composable and efficient $\mathbb{F}_{2^8}$ inverter implementations that achieve a lower area footprint and reduced randomness consumption, while maintaining the same latency as HPC3-based designs. Although this technique may not be trivially applicable to all algorithms, it represents a promising approach for developing implementations with arbitrary protection order and improved efficiency in both area and randomness.

First, we revisit the security of individual DOM multipliers (including DOM-*dep* and DOM-*indep*) under the glitch-extended probing model. We then establish results on the compositional security of masked circuits composed exclusively of DOM multipliers and their SNI variants. In particular, we provide a sufficient condition for the secure composition of DOM multipliers.

Based on this condition, we construct d-NI implementations of the $\mathbb{F}_{2^8}$ inverter. Furthermore, by leveraging the technique of storing the normal basis representation of the plaintext state in dedicated registers, as introduced in [GKM+24], we design more efficient round-based AES implementations that support arbitrary protection orders. As a side product, our analysis reveals practical attacks on AES implementations that incorrectly apply DOM-*dep* multipliers [GMK16b], which, to the best of our knowledge, have not been explicitly identified in prior work.

Finally, we present synthesis results of our constructions and compare them with state-of-the-art implementations supporting arbitrary protection orders. The results demonstrate that our designs achieve improved efficiency in terms of area and randomness consumption, while maintaining the same latency as HPC3-based implementations.

2 Preliminaries

2.1 Notations

In this work, we use lowercase Latin letters (e.g., x, y, z) to denote random variables in $\mathbb{F}_{2^n}$, where n is a positive integer. The bold lowercase Latin letters (e.g., $\boldsymbol{x}, \boldsymbol{y}$) represent $d+1$ shares of random variables, i.e., $\boldsymbol{x} = (x_0, x_1, \ldots, x_d)$, with $x = \sum_{i=0}^{d} x_i$. In particular, $\boldsymbol{z}, \boldsymbol{b}$ represent random vectors of lengths $\frac{d(d+1)}{2}$ and $d+1$, respectively, which correspond to the randomness used in the masked circuits. x_i^j refers to the j-th bit of the i-th share of x.

Additionally, capital Latin letters (e.g., X, Y, Z) are used to denote sets of random variables, while capital Greek letters (e.g., Ω, Ψ, Θ) represent sets of values. The assignment $X = \Omega$ indicates that the random variables in X take values from the set Ω, with $|\Omega| = |X|$. 2^X denotes for the power set of a set X.

2.2 Masking and Security Model

A masking transformation converts an unprotected cryptographic circuit C, which implements a Boolean function $f : \mathbb{F}_{2^m} \to \mathbb{F}_{2^n}$, into a shared circuit $\tilde{C}$ that implements a masked version of the function. The function f maps an input $x \in \mathbb{F}_{2^m}$ to an output $q \in \mathbb{F}_{2^n}$, i.e., $x \mapsto q$. Its shared form is denoted by $\boldsymbol{f} : (\mathbb{F}_{2^m}^{d+1}, \mathbb{F}_2^*) \to \mathbb{F}_{2^n}^{d+1}$, which maps a pair $(\boldsymbol{x}, \boldsymbol{z})$ to a shared output $\boldsymbol{q}$, i.e., $(\boldsymbol{x}, \boldsymbol{z}) \mapsto \boldsymbol{q}$. Here, $\boldsymbol{x} = (x_i)_{0 \le i \le d}$ and $\boldsymbol{q} = (q_i)_{0 \le i \le d}$ are the shared input and output, respectively. The original (unshared) variables are recovered by recombining the shares as $x = \bigoplus_i x_i$ and $q = \bigoplus_i q_i$. The vector $\boldsymbol{z}$ contains random values used to mask intermediate computations, ensuring they remain statistically independent of the secret input x.

An adversary is allowed to place d probes on the wires of the masked circuit $\tilde{C}$. By doing so, they can observe the joint probability distribution of d intermediate variables $w_1, \ldots, w_d$ carried by those wires. If every such distribution is statistically independent of the secret inputs, then the masked circuit $\tilde{C}$ is considered secure against such an adversary. This security model is known as the *probing model*, introduced in 2003 [ISW03].

However, in hardware implementations, *glitches* may occur due to signal transitions, potentially leaking more information. In this setting, the adversary is assumed to obtain full access to the joint distribution of the stable signals contributing to the computation of $w_1, \ldots, w_d$. These signals are denoted by $\bigcup_{1 \le i \le d} \rho(w_i)$, where ρ is a function mapping each intermediate variable to the set of stable signals (typically stored in registers) that influence it.

This security notion is referred to as the *glitch-extended probing model* [FGP+18]. As a special case, in the standard probing model, ρ is simply the identity function, i.e., $\rho(w_i) = \{w_i\}$. We refer to ρ as the *leakage function*, and to $\rho(w_i)$ as the *probing set* of w_i.

Definition 1 (Probing Security)**.** A masked circuit $\tilde{C}$ is said to provide d-probing security if, for any probe set $P = \{w_1, \ldots, w_t\}$ with $t \le d$, the joint probability distribution of the set of leaked signals $\bigcup_{1 \le i \le t} \rho(w_i)$ is statistically independent of the secret input x.

2.3 Circuit Composition

The simple composition of two probing-secure gadgets does not necessarily yield a secure implementation under the probing model. This limitation applies to both the standard probing model and its glitch-extended variant [CPRR13, BBD+16, MMSS19]. To address these composability issues, several composable security notions have been proposed in recent years.

In the following, we introduce the concept of simulation and the security notions that facilitate secure circuit composition, namely NI/SNI [BBD+16] and PINI [CS20].

The original definition of SNI is restricted to single-output functions, which limits its applicability to functions over the same field or ring. However, single outputs over larger fields can often be decomposed into multiple elements over smaller fields and used as inputs to other functions. This behavior cannot be captured by the single-output SNI definition, motivating our adoption of the multi-output variant of SNI (MO-SNI) proposed by Cassiers in [CS20].

Before presenting these compositional security notions, we first provide the necessary background on the simulation paradigm and the definition of Non-Interference, which form the foundation of SNI and PINI.

Definition 2 (Simulation [HB25])**.** Let X and O be two sets of random variables, and let $S \subseteq X$ with $\bar{S} = X \setminus S$. We say that S perfectly simulates the observations in O under X if and only if $\forall \Omega, \Psi, \Theta : \Pr[O = \Omega \mid S = \Psi] = \Pr[O = \Omega \mid S = \Psi, \bar{S} = \Theta]$.

Define $I_i^{t_1} = \{S | S \in 2^{\{x_j^i | 0 \le j \le d\}}, |S| = t_1\}$ for $0 \le i \le m-1$, then $S \in I_i^{t_1}$ is a set composed of exactly t_1 shares of sensitive variable x^i. Similarly, define $O_i^{t_2} = \{S | S \in 2^{\{q_j^i | 0 \le j \le d\}}, |S| = t_2\}$ for $0 \le i \le n-1$. Let $I_\times^{t_1} = \{\bigcup_{i=0}^{m-1} S_i \mid S_i \in I_i^{t_1} \text{ for } 0 \le i \le m-1\}$ (resp. $O_\times^{t_2} = \{\bigcup_{i=0}^{n-1} S_i \mid S_i \in O_i^{t_2} \text{ for } 0 \le i \le n-1\}$), then $I_\times^{t_1}$ (resp. $O_\times^{t_2}$) contains all possible sets with exactly t_1 (resp. t_2) shares of each bit of the input x (resp. output q).

Definition 3 (d-Non-Interference (NI) [BBD+16])**.** A masked circuit $\tilde{C}$ provides d-Non-Interference iff for any probe set $P = \{w_1, \ldots, w_t\}$ with $t \le d$, there exists a set $S \in I_\times^t$ such that $\bigcup_{1 \le i \le t} \rho(w_i)$ can be perfectly simulated by a subset of S.

Definition 4 (Multi-Output Strong Non-Interference [BBD+16])**.** A masked circuit $\tilde{C}$ provides d-Strong Non-Interference iff for any probe set P of size t_1 and any $O \in O_\times^{t_2}$, with $t = t_1 + t_2 \le d$, there exists a simulation set $S \in I_\times^{t_1}$ such that $\bigcup_{w_i \in P \cup O} \rho(w_i)$ can be perfectly simulated by a subset of S.

Define $I^t = \{S | S \in 2^{\{x_j | 0 \le j \le d\}}, |S| = t\}$ and $O^{t_2} = \{S | S \in 2^{\{q_j | 0 \le j \le d\}}, |S| = t_2\}$, then PINI can be defined as follows.

Definition 5 (Probe-Isolating Non-Interference [CS20])**.** A masked circuit $\tilde{C}$ provides d-Probe-Isolating Non-Interference iff for any probe set P of size t_1 and any $O \in O^{t_2}$, with $t = t_1 + t_2 \le d$, there exists a simulation set $S \in I^t$, such that $\bigcup_{w_i \in P \cup O} \rho(w_i)$ can be perfectly simulated by a subset of S. Moreover, the index set of output probes O must be a subset of index set of simulation set S.

2.4 Domain-Oriented Masking

Domain-Oriented Masking (DOM) was introduced by Gross *et al.* in 2016 [GMK16b, GMK17]. In their ePrint paper [GMK16b], they proposed two variants of masked multipliers, referred to as DOM-*indep* and DOM-*dep*. The DOM-*indep* variant assumes that the two input sharings are independent, whereas DOM-*dep* was intended to support cases with dependent inputs. However, Moos *et al.* later showed in TCHES 2018 that DOM-*dep* is vulnerable to a $(\lceil \frac{d}{2} \rceil + 1)$-order attack when the input sharings of the multiplicands are identical, for security orders $d \ge 2$.

In this work, we consider a tweaked version of DOM multipliers that integrate a commutative linear function $L(x, y)$ into the multiplication, resulting in the operation $xy + L(x, y)$.[1] We refer to x and y as the operands of the multiplication function.

Algorithm 1 and Algorithm 2 present tweaked versions of the original DOM-*indep* and DOM-*dep* gadgets, respectively. In the original constructions, $L(x, y) \equiv 0$. Register stages are marked as $\texttt{Reg}(\cdot)$. If the outputs of DOM-*indep* and DOM-*dep* are stored in registers, we denote the gadgets DOM-*indep*-SNI and DOM-*dep*-SNI, respectively.

[1] In the case of DOM-*dep*, certain security issues can be mitigated by swapping the operands connected to its two input ports. If the linear function L is commutative, such swapping does not affect the result.

2.5 $\mathbb{F}_{2^8}$ Inverter Decomposition

The decomposition of the $\mathbb{F}_{2^8}$ inverter used in this work is based on two existing designs: Canright's inverter [Can05] and Hadzic's inverter [HB25]. Canright's construction has a multiplicative depth of 4, while Hadzic's achieves a depth of 3, making the latter more suitable for low-latency masking.

Algorithm 1: DOM-*indep* multiplication with $d+1 \geq 2$ shares.

Input: Shares $\boldsymbol{x} = (x_i)_{0\leq i\leq d}$ and $\boldsymbol{y} = (y_i)_{0\leq i\leq d}$, such that $\oplus_i x_i = x$ and $\oplus_i y_i = y$. Masks $\boldsymbol{z} = (z_{ij})_{0\leq i<j\leq d}$.
Output: shares $\boldsymbol{q} = (q_i)_{1\leq i\leq d}$, such that $\oplus_i q_i = q = x \cdot y + L(x, y)$

Gadget DOM-*indep*$(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z})$**:**
- **for** $i = 0$ **to** d **do**
  - **for** $j = i + 1$ **to** d **do**
    - $z_{ji} = z_{ij}$;
- **for** $i = 0$ **to** d **do**
  - $t_{ii} = \mathrm{Reg}(x_i y_i + L(x_i, y_i))$;
  - **for** $j = 0$ **to** d **with** $j \neq i$ **do**
    - $t_{ij} = \mathrm{Reg}(x_i y_j \oplus z_{ij})$;
  - $q_i = \sum_j t_{ij}$;
- **return** $\boldsymbol{q} = (q_i)_{0\leq i\leq d}$;

Algorithm 2: DOM-*dep* multiplication with $d+1 \geq 2$ shares.

Inputs: Shares $\boldsymbol{x} = (x_i)_{0\leq i\leq d}$ and $\boldsymbol{y} = (y_i)_{0\leq i\leq d}$, such that $\oplus_i x_i = x$ and $\oplus_i y_i = y$. Masks $\boldsymbol{z} = (z_{ij})_{0\leq i<j\leq d}$ and $\boldsymbol{b} = (b_i)_{0\leq i\leq d}$.
Outputs: shares $\boldsymbol{q} = (q_i)_{1\leq i\leq d}$, such that $\oplus_i q_i = q = x \cdot y + L(x, y)$

Gadget DOM-*dep*$(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z}, \boldsymbol{b})$**:**
- **for** $i = 0$ **to** d **do**
  - $u_i = \mathrm{Reg}(y_i \oplus b_i)$;
- $u = \sum_{i=0}^{d} u_i$;
- **for** $i = 0$ **to** d **do**
  - **for** $j = i + 1$ **to** d **do**
    - $z_{ji} = z_{ij}$;
- **for** $i = 0$ **to** d **do**
  - $t_{ii} = \mathrm{Reg}(x_i b_i + L(x_i, y_i))$;
  - **for** $j = 0$ **to** d **with** $j \neq i$ **do**
    - $t_{ij} = \mathrm{Reg}(x_i b_j \oplus z_{ij})$;
  - $v_i = \sum_j t_{ij}$;
- **for** $i = 0$ **to** d **do**
  - $q_i = \mathrm{Reg}(x_i)u + v_i$;
- **return** $\boldsymbol{q} = (q_i)_{0\leq i\leq d}$;

Figure 1 and Figure 2 show tweaked versions of Canright's and Hadzic's S-Boxes, respectively. In these versions, the multipliers and squarer-scalers over $\mathbb{F}_{2^2}$ and $\mathbb{F}_{2^4}$ are merged into unified operations within their respective fields. These merged components are labeled as Sq. Sc. Mult. $\mathbb{F}_{2^2}$ and Sq. Sc. Mult. $\mathbb{F}_{2^4}$ in the figures. This merge absorbs

the linear operation into the multiplications, making the decomposition of the $\mathbb{F}_{2^8}$ inverter consist solely of the multiplicative functions targeted by DOM multipliers, without any separate non-trivial linear components[2].

The notations $(\cdot)^{-1}$ and $(\cdot)^2$ in the figures represent inversion and squaring over $\mathbb{F}_{2^2}$, which correspond to swapping the $\mathbb{F}_2$ components. Similarly, $(\cdot)^4$ denotes a squaring over $\mathbb{F}_{2^4}$, which swaps the $\mathbb{F}_{2^2}$ components. The symbol ⧺ indicates the concatenation of two $\mathbb{F}_{2^n}$ elements into a single element in $\mathbb{F}_{2^{2n}}$.

In both figures, X_0 and X_1 denote the four least significant bits (LSBs) and most significant bits (MSBs) of the input, respectively. In Figure 1, Y_0 and Y_1 denote the four LSBs and MSBs of the output. In contrast, Figure 2 uses Y_0, Y_1, Y_2, and Y_3 to represent the output bits in order from the two LSBs to the two MSBs.

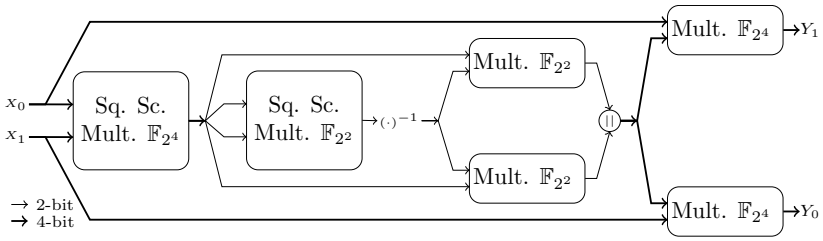

Figure 1: Decomposition of Canright's $\mathbb{F}_{2^8}$ Inverter

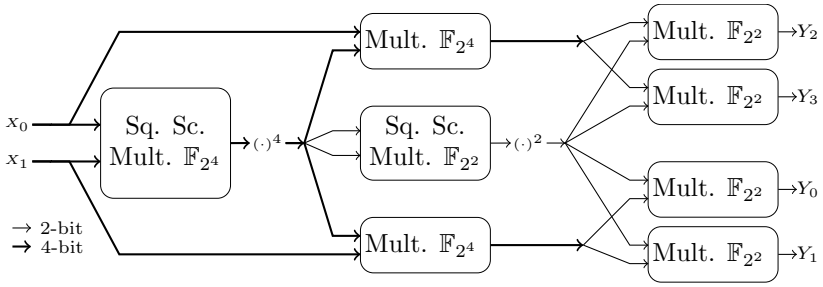

Figure 2: Decomposition of Hadzic's $\mathbb{F}_{2^8}$ Inverter

3 Security Analysis of DOM

In this section, we will give security analysis of the DOM multipliers. Firstly, we will give a more detailed analysis for the ability of the adversary. Secondly, we provide a security analysis of individual DOM multipliers, including DOM-*dep*, DOM-*indep*, and their SNI variants. Finally, we will give security analysis of a masked circuit exclusively composed of the DOM multipliers introduced in Sect. 2.4 without any non-trivial linear mapping.

3.1 Introduction to DOM multipliers

In domain-oriented masking, each share of a variable is associated with a domain. The basic idea of the DOM approach is to keep the shares of all domains independent from shares of other domains.

In this work, the variables are $d+1$ shared, thus there are $d+1$ domains. We use the share indices to denote domains, i.e., share x_i with subscript i is in domain i.

[2]Trivial linear mapping includes identity mapping and permutation, *etc.*

Figures 3 and 4 respectively illustrate the d-th order DOM-*indep* and DOM-*dep* multipliers, which implement the function $q = x \cdot y + L(x, y)$. The integration of the linear terms into the multiplier is straightforward: each linear component in domain i is simply added to the corresponding inner domain term i.e., $x_i y_i$ and $x_i b_i$.

Register stages are marked with an triangle and a dashed gray line. According to the terms stored in registers, the registers in DOM-*indep* can be classified into two groups: (i) inner domain terms for $\boldsymbol{x}$ and $\boldsymbol{y}$, i.e., $x_i y_i + L(x_i, y_i)$; (ii) cross domain terms for $\boldsymbol{x}$ and $\boldsymbol{y}$, i.e., $x_i y_j + z_{ij}$. There are four groups of registers in DOM-*dep*: (i) pipeline registers for $\boldsymbol{x}$, i.e., $x_{i,.}$; (ii) inner domain terms for $\boldsymbol{x}$ and $\boldsymbol{b}$, i.e., $x_i b_i + L(x_i, y_i)$; (iii) cross domain terms for $\boldsymbol{x}$ and $\boldsymbol{b}$, i.e., $x_i b_j + z_{ij}$; (iv) blinded $\boldsymbol{y}$ terms, i.e., $y_{i,.} + b_{i,.}$.

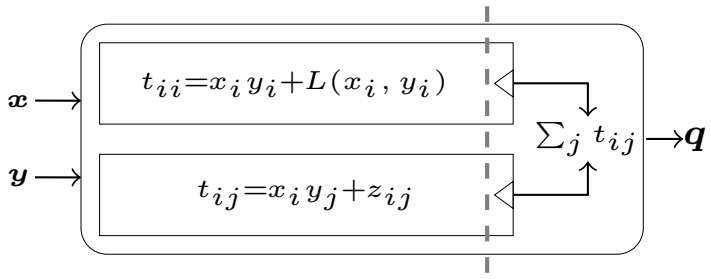

Figure 3: DOM-*indep* multiplier where $z_{ij} = z_{ji}$, $i \neq j$ and $0 \leq i, j \leq d$, and $q_i = \sum_j t_{ij}$.

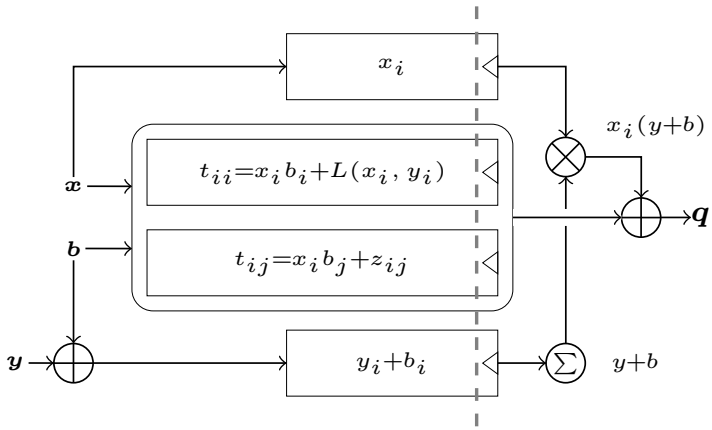

Figure 4: d-th order DOM-*dep* multiplier where $z_{ij} = z_{ji}$, $i \neq j$ and $0 \leq i, j \leq d$. The summation of t_{ij} with $0 \leq j \leq d$ is omitted in this figure to save space. $q_i = \sum_j t_{ij} + x_i(y + b)$.

3.2 Field-level Probing and Bit-level Probing

In this work, we employ two types of probes: field-level probes and bit-level probes. Bit-level probes are necessary in certain cases, such as when analyzing MO-SNI or the composition of DOM multipliers. In other scenarios, field-level probes are preferred as they enhance clarity and improve the readability of the paper.

We use an example to illustrate the difference between field-level probes and bit-level probes.

We observe that when a probe is placed at an output wire of a unmasked field multiplication in $\mathbb{F}_{2^n}$, the adversary will learn all bits of the operands of this multipliers.

This is because each output bit of the field multiplication depends on all bits of both operands. However, when a probe is placed at the result of a XOR operation, the adversary might only learn one bit of a field elements. For example, a probe placed at the k-th bit of t_{ij},i.e., $t_{ij}^k = (x_i y_j)^k + z_{ij}^k$ ($x_i, y_j, z_{ij} \in \mathbb{F}_{2^n}$ are stored in registers) will enable the adversary to access all bits in x_i and y_j, but only the k-th bit of z_i since the k-th bit of $x_i y_j$ depend on all bits of x_i and y_j while the bit-wise XOR only depends on z_{ij}^k. To avoiding this inconsistency, we assume the adversary can get access to all bits of z_{ij}, i.e., when the adversary gets a bit of a field element, he automatically gets the other bits of the same field element. However, when proving that a masked circuit is d-MO-SNI and the adversary is placing probes on the output q, this probing ability is disabled and the adversary can only use probes at bit-level. If the adversary places a probe on the q_i^j,i.e., i-th share of the j-th output bit, he can only learn q_i^j rather than q_i. Otherwise, we can only prove the masked circuit is d-SNI with q being the single-output rather than d-MO-SNI with bits in q being multiple outputs.

On the other hand, if a bit is computed by the other bits through combinational logic, then the glitch-extended leakage function of this bit is the union set of glitch-extended leakage function of these bits. For example, $\rho(t_{ij}^k) = \rho(x_i) \cup \rho(y_j) \cup \rho(z_{ij})$ for $t_{ij} = x_i y_j + z_{ij}$. For brevity, $\rho(t_{ij}^k)$ can be written as $\rho(x_i), \rho(y_j), z_{ij}$. Moreover, the glitch-extended probing set of x_i is the union set of the glitch-extended probing set of the bits in x_i, i.e., $\rho(x_i) = \bigcup_{0 \le j \le n-1} \rho(x_i^j)$.

Moreover, we would like to grant the adversary stronger ability to access the distribution of the observations. When he probes the output q_i of the non-SNI DOM multipliers, he can get the distribution of $t_{ii} = x_i b_i + L(x_i, y_i)$ or $t_{ii} = x_i y_i + L(x_i, y_i)$. In this case, we would like he obtain the joint distribution of x_i, y_i, b_i in case of DOM-*dep* and x_i, y_i in case of DOM-*indep* instead of just t_{ii}. This enhance the ability of the adversary since the adversary can derive the distribution of t_{ii} from the distribution of x_i, y_i (and b_i), but he can not derive the joint distribution of x_i, y_i (and b_i) from the distribution of t_{ii}.

As a consequence, $\rho(q_i)$ can be rewritten as $\{x_i, y_i, b_i\} \cup \{y_j + b_j | 0 \le j \le d, j \ne i\} \cup \{t_{ij} | 0 \le j \le d, j \ne i\}$ for DOM-*dep*, $\rho(q_i)$ can be rewritten as $\{x_i, y_i\} \cup \{t_{ij} | 0 \le j \le d, j \ne i\}$ for DOM-*indep*.

We partition glitch-extended probing set of a wire w into three disjoint sets, *shares, masked expressions*, and *masks*. And we define the functions $\delta(w), \mu(w), \eta(w)$ which map wire w to the subset of $\rho(w)$ consists of *shares, masked expressions*, and *masks*, respectively.

Let π be a function that maps an output q to the set of sensitive variables whose shares in the same domains can be inferred when the adversary places $t \le d$ probes on t shares of q. For both DOM-*dep* and DOM-*indep*, we have $\pi(q) = \{x, y\}$, while for DOM-*dep*-SNI and DOM-*indep*-SNI, $\pi(q) = \{q\}$.

3.3 Security analysis of a single DOM multiplier

Gross *et al.* proposed the DOM-*dep* multiplier in their eprint paper [GMK16b] for the purpose of dealing with dependent inputs even if the input sharing is identical. Later in 2018, as shown by Moos *et al.* [MMSS19], this security claim does not hold true for identical input sharing when security order is greater than 2.

In this section, we consider the security of DOM multipliers in the condition that the two inputs to the multiplier are shared independently.

3.3.1 Security analysis of first-order DOM-*dep* multipliers.

Gross *et al.* provides a first-order optimization to DOM-*dep*. The two shares of output $q = xy$ can be written as $q_0 = [x_0]([y_0]+[y_1+b_0])+[x_0 b_0+z_0], q_1 = [x_1]([y_1]+[y_0+b_0])+[x_1 b_0+z_0]$. In fact, it is the same as the first order case of the HPC3.1 multiplier proposed in [HB25]

with the field $\mathbb{F}_p$ being $\mathbb{F}_{2^n}$. As a result, it fulfills the 1-PINI property in the glitch-extended probing model and is trivially composable.

In the following, we ignore the first optimization to DOM-*dep*, and discuss the regular DOM-*dep* multiplier for $d \geq 1$.

3.3.2 Security analysis of higher-order DOM multipliers.

When doing security analysis under glitch-extended probing model, it suffices to only consider the combination of d probes that are all placed at the input wires of the registers (and/or the output wires of a masked circuit). It is a widely employed strategy in the previous work [FGP+18, CGLS21] and the field of formal verification of hardware masking [BBC+19, KSM20, ZCF25] since the maximal leakage sets are always at the input of the registers. Additionally, it is perfectly fine to ignore the pipeline registers such as x_i Fig. 4 since the observations it leaks are only a subset of input wires of other registers, such as the ones storing t_{ii}, t_{ij}. We assume the adversary probes at field-level in Subsubsection 3.3.2.

Proposition 1. *DOM-dep is glitch-robust d-NI.*

Our proof is similar to the simulation-based proofs in [FGP+18, CGLS21]. While simulation-based proof technique only involves with shares of the original sensitive variables, our proof technique also deals with intermediate sensitive variables when considering the composition of DOM multipliers, as shown in Subsection 3.4.

Process I: Domain Recovery Process for a probe placed at DOM-*dep*

(1) If the probe is placed at t_{ii}, add x_i, y_i into $\mathcal{S}$, add b_i into $\mathcal{R}$. If $i \notin I_x$, add i to I_x. If $i \notin I_y$, add i to I_y.

(2) If the probe is placed at t_{ij}, add x_i into $\mathcal{S}$, add z_{ij}, b_j into $\mathcal{R}$. If $i \notin I_x$, add i to I_x. If $j \notin I_y$ and $y_j + b_j \in \mathcal{O}$, add j to I_y. **If** $x_j b_i + z_{ij}, y_i + b_i \in \mathcal{O}$ **and** $i \notin I_y$**, add** i **to** I_y.

(3) If the probe is placed at $y_i + b_i$, add y_i into $\mathcal{S}$, add b_i into $\mathcal{R}$. If $i \notin I_y$, add i to I_y.

(4) If the probe is placed at q_i, add x_i, y_i into $\mathcal{S}$, add $(x_i b_k + z_{ik})_{i \neq k, 0 \leq k \leq d}, (y_k + b_k)_{i \neq k, 0 \leq k \leq d}$ into $\mathcal{E}$, add b_i into $\mathcal{R}$. If $i \notin I_x$, add i to I_x. If $i \notin I_y$, add i to I_y. **Let** $K_1 = \{k | z_{ik} \in \mathcal{O}, k \notin I_y, 0 \leq k \leq d\}$ **and** $K_2 = \{k | b_k \in \mathcal{O}, k \notin I_y, 0 \leq k \leq d, k \neq i\}$**. For all** $k \in K_1 \cup K_2$**, add** k **to** I_y**.**

Proof. In our proof, the adversary maintains a record of the observations $\mathcal{O}$ collected by placing probes. The set $\mathcal{O}$ is partitioned into three disjoint subsets: (1) $\mathcal{S}$: *shares* of x and y; (2) $\mathcal{E}$: *masked expressions*, such as $b_i + y_i$ or $x_i b_j + z_{ij}$; (3) $\mathcal{R}$: *masks*, such as b_i and z_{ij}.

The adversary also maintains two sets of domain indices, I_x and I_y, representing the recovered domains of x and y, respectively.

Without loss of generality, we assume that the adversary places probes only at the input wires of registers storing $t_{ii}, t_{ij}, y_i + b_i$, and the output wires q_i, where $0 \leq i, j \leq d$ and $i \neq j$.

Initially, the adversary sets $I_x = I_y = \emptyset$. For each probe placed, they follow Process I to record both the observations and the recovered domains. We show that this process accurately tracks the observed values as well as the recovered domains of x and y. Moreover,

after placing t probes, it holds that $|I_x| \leq t$ and $|I_y| \leq t$ for any $t \leq d$, which implies Proposition 1.

Base case: When $t = 0$, no probes are placed, and no domain of any sensitive variable is recovered. The claim trivially holds.

Inductive step: Assume the claim holds for $t = n$. We show it also holds for $t = n + 1$, where $0 \leq n \leq d - 1$.

Let $\mathcal{S}^t$, $\mathcal{E}^t$, and $\mathcal{R}^t$ be the partition of the observation set $\mathcal{O}$ after the adversary has placed t probes. Let $\mathcal{P}$ denote the observations from the (t+1)-th probe, and let $\mathcal{P}_s$, $\mathcal{P}_e$, and $\mathcal{P}_r$ be its partition into *shares*, *masked expressions*, and *masks*, respectively.

There are only three ways for the adversary to recover domains of sensitive variables: (1) directly from observed shares; (2) by using known masks in $\mathcal{R}$ to unmask expressions in $\mathcal{E}$; (3) by identifying multiple masked expressions in $\mathcal{E}$ that share the same mask.

The third case never yields new domain information. The only scenario where multiple expressions use the same mask is with $x_i b_j + z_{ij}$ and $x_j b_i + z_{ij}$ sharing z_{ij}. However, according to the adversary's procedure, $x_i b_j + z_{ij}$ is added to $\mathcal{O}$ only if q_i is probed, and similarly $x_j b_i + z_{ij}$ only if q_j is probed. After placing these two probes, both indices i and j are included in I_x and I_y, meaning that combining these expressions provides no further domain recovery.

Therefore, when the adversary obtains new observations $\mathcal{P}$ from the newly placed (t+1)-th probe, they only need to consider the first two recovery cases discussed previously.

First, the adversary updates the recovered domains using $\mathcal{P}_s$. Second, they check: (1) whether any mask in $\mathcal{P}_r$ can unmask a masked expression in $\mathcal{E}^t$ (we do not need to consider $\mathcal{P}_e$ here, as these expressions have already been accounted for by the glitch-extended leakage function); (2) whether any mask in $\mathcal{R}^t$ can unmask an expression in $\mathcal{P}_e$ (we do not need to revisit $\mathcal{E}^t$ since it has been processed by the previous t probes).

For probe groups (1) and (3) in Process I, domain i is potentially added to I_x and I_y (or only I_y) through $\mathcal{P}_s = \{x_i, y_i\}$ and $\mathcal{P}_s = \{y_i\}$, respectively. In both cases, we have $\mathcal{P}_e = \emptyset$ and $\mathcal{P}_r = \{b_i\}$. The mask b_i can only unmask $y_i + b_i$ to recover y_i, which is already recovered via $\mathcal{P}_s$. Thus, b_i provides no additional domain information. Consequently, the increase in recovered domains for x and y is solely due to $\mathcal{P}_s$, and remains at most t+1.

For probe group (2), i is possibly added to I_x via $\mathcal{P}_s = \{x_i\}$, increasing $|I_x|$ by at most one. Regarding y, we note that $b_j \in \mathcal{P}_r$ might unmask $y_j + b_j$, and $z_{ij} \in \mathcal{P}_r$ might unmask $x_j b_i + z_{ij}$, provided $y_j + b_j$ and $x_j b_i + z_{ij}$ exist in $\mathcal{O}$. At first glance, this seems to allow up to two new domains of y to be recovered. However, if i is newly added to I_y, then j must have already been included in I_y via a previous probe at q_j, which introduced $x_j b_i + z_{ij}$. Hence, at most one new domain is added to I_y in this case. Therefore, the total increase in recovered domains for x and y is still at most one, keeping the count below or equal to t+1.

For probe group (4), i may be added to both I_x and I_y via $\mathcal{P}_s$. The analysis of $b_i \in \mathcal{P}_r$ parallels that of groups (1) and (3). We now analyze whether masks in $\mathcal{R}^t$ can unmask any expression in $\mathcal{P}_e$: specifically, expressions of the form $(x_i b_k + z_{ik})_{i \neq k}$ or $(y_k + b_k)_{i \neq k}$.

Indeed, the random masks in $K_1 \cup K_2$ (defined in Process I(4)) can be used to recover multiple new domains of y. However, there exist corresponding prior probes—$|K_1|$ of them at inputs of $\{x_k b_i + z_{ik} \mid k \in K_1\}$—which previously added at most domain i or none at all to I_y. Similarly, $|K_2|$ probes were placed at $\{x_m b_k + z_{mk} \mid k \in K_2\}$, which added nothing to I_y. Hence, the number of y-domains recovered by the first t probes is at most $(t - |K_1| - |K_2| + 1)$[3]. After placing the (t+1)-th probe, up to $|K_1 \cup K_2| \leq |K_1| + |K_2|$ new domains are added to I_y, giving a total of at most t+1 recovered domains.

Therefore, in all cases, the number of recovered domains for x and y is no more than t+1 after placing the (t+1)-th probe.

[3]The '+1' accounts for the current probe possibly adding i if it wasn't previously added.

This completes the induction, and thus DOM-*dep* is d-NI under the glitch-extended probing model. □

Proposition 2. *DOM-indep is glitch-robust d-NI.*

Proposition 2 was originally proven in [CGLS21]. For the sake of completeness, we include a proof in Appendix B.1. However, we present the adversary's procedure—Process II—here for later reference.

Process II: Domain Recovery Process for a probe placed at DOM-*indep*

(1) If a probe is placed at t_{ii}, add x_i, y_i into $\mathcal{S}$. If $i \notin I_x$, add i to I_x. If $i \notin I_y$, add i to I_y.

(2) If a probe is placed at t_{ij}, add x_i, y_j into $\mathcal{S}$, add z_{ij} into $\mathcal{R}$. If $i \notin I_x$, add i to I_x. If $j \notin I_y$, add j to I_y. **If** $x_j y_i + z_{ij} \in \mathcal{O}$ **and** $i \notin I_y$**, add** i **to** I_y.

(3) If a probe is placed at q_i, add x_i, y_i into $\mathcal{S}$, add $(x_i y_k + z_{ik})_{i \neq k, 0 \leq k \leq d}$ into $\mathcal{E}$. If $i \notin I_x$, add i to I_x. If $i \notin I_y$, add i to I_y. **Let** $K_1 = \{k \mid z_{ik} \in \mathcal{O},\ k \notin I_y,\ 0 \leq k \leq d\}$**. For all** $k \in K_1$**, add** k **to** I_y**.**

Note that in probe group (4) of Process I and probe group (3) of Process II, the statements in bold ensure that the recovered domains of x and y are aligned by the $|K_1|+1$ or $|K_1 \cup K_2| + 1$ probes. As a result, after these probes are placed, the recovered domains of x and y by these probes (excluding the rest probes) become identical. Similarly, probe group (2) in both Process I and Process II aligns the recovered domains of x and y using two probes.

This form of domain recovery through alignment does not compromise security, even when the shares of x and y are identical, because the adversary learns the same domains of x and y.

We distinguish between two types of domain recovery. If the recovered domain of a sensitive variable is consistent with the index when it is probed, we refer to it as *direct domain recovery*. If the recovered domains of a sensitive variable did not include the index when it is probed, refer to it as *aligned domain recovery*. Note that, in both Process I and Process II, a probe can only result in either direct or aligned domain recovery for a sensitive variable, but not both simultaneously.

3.4 Compositional security analysis of DOM multipliers

Most existing works focus on Strong Non-Interference (SNI) within the same field or ring. For example, masked AES S-Box software implementations typically realize the inversion over $\mathbb{F}_{2^8}$ using exponentiation to the power of 254, with all operations performed in the field $\mathbb{F}_{2^8}$ [BBD+16, CS20]. Other masked implementations [GR17, BGR18] adopt the bit-level Boyar-Peralta representation [BP12], which operates solely over $\mathbb{F}_2$.

However, cryptographic algorithms may involve computations across multiple fields. For example, Canright's S-Box [Can05] employs algebraic operations over $\mathbb{F}_2$, $\mathbb{F}_{2^2}$, and $\mathbb{F}_{2^4}$. Furthermore, an element in $\mathbb{F}_{2^{2n}}$ can be represented as a pair of elements in $\mathbb{F}_{2^n}$, which can then be treated as two operands of a multiplier in $\mathbb{F}_{2^n}$.

Consider a 1-SNI multiplication in $\mathbb{F}_{2^2}$: $(b, a) \times (d, c) = (y, x)$, where (b, a) and (d, c) are the inputs, and (y, x) is the output. If the least significant bit x is subsequently multiplied with the most significant bit y, the single-output SNI notion becomes insufficient. Specifically, single-output SNI guarantees that each share of the output—namely, (y_0, x_0)

and (y_1, x_1)—is independent of any share of the inputs. However, in the following 1-SNI multiplication $x \times y$, an intermediate term such as $x_0 y_1$ may be computed. This gives the adversary access to one domain associated with part of the bits of the field element (e.g., x_0), and another domain associated with different bits (e.g., y_1). This situation is not covered by the single-output SNI definition, which only considers share combinations like (x_0, y_0) and (x_1, y_1).

Nevertheless, the composition of $(b, a) \times (d, c)$ followed by $x \times y$ remains secure, even though it cannot be verified under the single-output SNI framework. This limitation motivates the need for a multi-output version of SNI. Fortunately, Cassiers *et al.* have already introduced such variants in [CS20], namely Multi-Output SNI (MO-SNI) and Multi-Input Multi-Output SNI (MIMO-SNI).

Since an (MO-)SNI circuit that applies a linear transformation to its inputs generally does not retain the (MO-)SNI property, the authors proposed the stronger MIMO-SNI notion to enable trivial composition. This, however, imposes stricter constraints on the simulator when input shares are involved. In our case, the tweaked version of $\mathbb{F}_{2^8}$ does not involve any non-trivial linear map. Hence, we adopt the MO-SNI notion in this work.

Proposition 3. *Both DOM-dep and DOM-indep are glitch-robust d-MO-SNI if the outputs are stored in registers.*

In the following proof of Proposition 3, we assume that the adversary does not place probes at the field level when observing output shares. If field-level probes are allowed on output shares, we can only prove that DOM-*dep* and DOM-*indep* achieve SNI in $\mathbb{F}_{2^n}$, rather than MO-SNI over $\mathbb{F}_2$.

Proof. Whether considering DOM-*dep* or DOM-*indep*, the output shares stored in registers are of the form $q_i = x_i y + \sum_{j \neq i} z_{ij}$, where $x, y, z, q \in \mathbb{F}_{2^n}$. Therefore, the i-th share of the k-th bit of the output, denoted q_i^k, is masked by d random bits, namely $\{z_{ij}^k \mid j \neq i,\ 0 \leq j \leq d\}$. Note that for each bit of $q \in \mathbb{F}_{2^n}$, a distinct group of $\frac{d(d+1)}{2}$ random bits is used for protection.

Let us first consider the t_2 shares of a single output bit u. Each share u_i is protected by d random bits $\{z'_{ij} \mid j \neq i,\ 0 \leq j \leq d,\ z'_{ij} \in \mathbb{F}_2\}$. Suppose the indices of these t_2 output shares form the set $I_2 = \{i_1, \ldots, i_{t_2}\}$. The remaining indices then form the set $I_1 = \{0, \ldots, d\} \setminus I_2$, with $|I_1| = d + 1 - t_2$.

For any $k \in I_1$, the random mask z'_{ik} appears only once among the t_2 observed output shares and is used solely to mask u_i. Therefore, considering only the t_2 output probes, each share u_i is protected by $d + 1 - t_2$ unique masks[4]. The t_1 internal probes can reveal at most t_1 of these masks, since each probe can expose at most one bit of the form z'_{ij} for the same output bit[5].

Consequently, there remains at least $(d + 1 - t_2) - t_1 \geq 1$ random bit that is neither revealed nor used elsewhere in $\mathcal{O}$, still protecting u_i. Thus, the distribution of u_i remains independent of any input share. The same argument applies to all other shares and all bits: under t_1 internal probes, each of the t_2 observed shares of any output bit remains masked by at least one random bit that is used solely for that share and that bit. Hence, the joint distribution of the t_2 shares of each output bit remains independent of any input shares, even in the presence of t_1 internal probes.

Since DOM-*dep* and DOM-*indep* are d-NI in the glitch-extended probing model, any t_1 internal probes can be simulated by t_1 shares of each input bit. Therefore, the joint distribution of the t_2 output shares (for each bit) together with the t_1 internal probes can be simulated using at most t_1 input shares per bit, provided that $t_1 + t_2 \leq d$. □

[4] An additional $t_2 - 1$ masks z'_{ik} with $k \in I_2$, $k \neq i$, are shared between u_i and u_k.

[5] It remains secure even if the adversary uses field-level probes on internal variables. While such probes can obtain $z_{ij} \in \mathbb{F}_{2^n}$, each bit of z_{ij} masks a different bit of $x_i y$, and only one of these random bits contributes to the masking of the bit u. Hence, only one relevant random bit is potentially revealed.

Since DOM-*dep* and DOM-*indep* can achieve MO-SNI in two cycles (denoted as DOM-*dep*-SNI and DOM-*indep*-SNI, respectively), they can be used to construct composable masked circuits across multiple fields with SNI refreshing. However, gadgets satisfying MO-SNI are generally inefficient in terms of both randomness usage and latency. In particular, their two-cycle latency is substantial compared to single-cycle HPC3 designs. In the following, we present results that allow for avoiding the storage of outputs in registers, thereby improving latency performance.

We restrict our attention to circuits exclusively composed of DOM multipliers (including DOM-*dep*, DOM-*indep*, DOM-*dep*-SNI, and DOM-*indep*-SNI), where connections between different DOM multipliers consist only of wires carrying intermediate results. Naturally, the masked circuit $\tilde{C}$ must not contain any cyclic data dependencies.

In Sect. 3.2, we introduced the function π for the unmasked outputs from DOM multipliers in $\tilde{C}$. We now extend this definition to apply to individual sensitive bits and to sets of sensitive bits, where each bit is either an unmasked output bit DOM multiplier or an input bit of C.

We begin by defining π for a single-bit variable u:

- If u is an input bit of the unmasked circuit C whose shares are stored in registers, then $\pi(u) = \{u\}$.
- If u is a bit of the output q of a DOM-*dep* or DOM-*indep* multiplier, then $\pi(u) = \pi(q)$.
- If u is a bit of the output q of a DOM-*dep*-SNI or DOM-*indep*-SNI multiplier, then $\pi(u) = \{u\}$.

Next, for a multi-bit variable x (viewed as a set of bits), we define $\pi(x) = \bigcup_{u \in x} \pi(u)$.

This extended definition is consistent with the one introduced in Sect. 3.2. In particular, the image of π for the output q of any DOM multiplier remains unchanged. The relationship between $\delta(x_i)$ and $\pi(x)$ is that $\delta(x_i)$ reveals domains i for variables in $\pi(x)$.

Proposition 4. *Let $\tilde{C}$ be a masked circuit exclusively composed of DOM multipliers with wires connecting*[6] *between the inputs and outputs of these multipliers (ignoring pipeline registers). Its corresponding unmasked circuit C has no cyclic data dependency. The inputs of $\tilde{C}$ is shared independently and stored in registers. If for all DOM-dep($\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z}, \boldsymbol{b}$) and DOM-dep-SNI($\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z}, \boldsymbol{b}$), $y \cap \pi(x) = \emptyset$*[7]*, and for all DOM-indep($\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z}$) and DOM-indep-SNI($\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z}$), $\pi(x) \cap \pi(y) = \emptyset$, then $\tilde{C}$ is glitch-robust d-NI. Here, each bit of the operands x, y of all multipliers is either an input bit of $\tilde{C}$, or an output bit of a DOM multiplier.*

Proof. The adversary constructs the set A of all sensitive variables in $\tilde{C}$, including the input bits of C, the output bits of each multiplication.

The adversary constructs a set I_u consists of recovered domains for each sensitive variable $u \in A$. When the number of probes is t, we denote the set with I_u^t. Whenever a domain i is recovered for the sensitive variable u, add i to I_u.

We refer to the multiplier in which a probe is internally placed as the *probed multiplier* (we assume the probed multiplier implements the multiplication function $xy + L(x, y)$ in the rest of the proof). A probe can reveal not only the domains of sensitive inputs or outputs of the probed multiplier, but also those of other multipliers whose outputs serve as inputs to the probed multiplier, due to the presence of glitches. We refer to these as *glitch-dependent multipliers*.

It suffices to consider that the adversary only places probes at the input wires of registers that store the masked computation of the form listed in column 2 of Table 1 where each of first five rows is from a certain DOM multipliers and the q_is in the last four rows are possible forms of the output shares of $\tilde{C}$.

[6] Of course, the wire connections should be within the same domain, which means the output share q_i of a multiplier is never connected to the input shares x_j or y_j of another multiplier with $j \neq i$.

[7] In this proposition, the operands x, y of multipliers can also be viewed as a set of bits.

Table 1: Possible probing positions and their dependency

No.	Probe Types	Shares to be extended	Extended Probes
1	$x_i y_i + L(x_i, y_i)$	x_i, y_i	$\rho(x_i), \rho(y_i)$
2	$x_i b_i + L(x_i, y_i)$	x_i, y_i	$\rho(x_i), \rho(y_i), b_i$
3	$x_i y_j + z_{ij}$	x_i, y_j	$\rho(x_i), \rho(y_j), z_{ij}$
4	$x_i b_j + z_{ij}$	x_i	$\rho(x_i), b_j, z_{ij}$
5	$y_i + b_i$	y_i	$\rho(y_i), b_i$
6	q_i (DOM-*indep*)	q_i	$x_i, y_i, (x_i y_k + z_{ik})_{k\neq i}$
7	q_i (DOM-*dep*)	q_i	$x_i, y_i, b_i, (y_k + b_k)_{k\neq i}$, $(x_i b_k + z_{ik})_{k\neq i}$
8	q_i (DOM-*indep*-SNI)	q_i	q_i
9	q_i (DOM-*dep*-SNI)	q_i	q_i

To prove proposition 4, it suffices to prove the conclusion that for any $0 \leq t \leq d$ probes and any $u \in A$, $|I_u^t| \leq t$.

We now give a formal proof using mathematical induction.

Base case: When $t = 0$, no probes are placed, and no domain of any sensitive variable is recovered. The claim trivially holds.

Inductive step: Assume the claim holds for $t = n$. We show it also holds for $t = n + 1$, where $0 \leq n \leq d - 1$.

As in the proof for Proposition 1, the adversary partition $\mathcal{O}^t$ into three disjoint subsets[8]: (1) $\mathcal{S}^t$ consists of *shares* of sensitive variables in A; (2) $\mathcal{E}^t$ consists of *masked expressions*; (3) $\mathcal{R}^t$ consists of *masks*.

Initially, the adversary set I_u to $\emptyset$ for all $u \in A$. Then they run the following algorithm for each probe placed inside $\tilde{C}$ to keep a record of observations and recovered domains for each sensitive variable. We refer the DOM multiplier that the $(t+1)$-th probe placed internally the current multiplier.

1. Update the recovered domains of sensitive input bits of $\tilde{C}$ or sensitive variables in glitch-dependent multipliers. For each bit u (indexed by $i' \in \{i, j\}$) in the variables listed in column 3 (Shares to be extended) of Table 1, perform the following:

   (1) if $u_{i'}$ is an output bit from a DOM-*indep*-SNI or DOM-*dep*-SNI multiplier, or if $u_{i'}$ is an input bit, then add $u_{i'}$ to the set $\mathcal{S}$;

   (2) if u is an output bit of a DOM-*dep* multiplier implementing $x'y' + L(x', y')$ with randomness $(z'_{kl})_{k\neq l,\ 0\leq k,l\leq d}$ and $(b'_k)_{0\leq k\leq d}$, invoke Process I(4) with parameters $q, x, y, i, z, b, \mathcal{O}, \mathcal{S}, \mathcal{R}, \mathcal{E}$ assigned as $u, x', y', i', z', b', \mathcal{O}, \mathcal{S}, \mathcal{R}, \mathcal{E}$;

   (3) if u is an output bit of a DOM-*indep* multiplier implementing $x'y'+L(x', y')$ with randomness $(z'_{kl})_{k\neq l,\ 0\leq k,l\leq d}$, invoke Process II(3) with parameters $q, x, y, i, z, \mathcal{O}, \mathcal{S}, \mathcal{R}, \mathcal{E}$ assigned as $u, x', y', i', z', \mathcal{O}, \mathcal{S}, \mathcal{R}, \mathcal{E}$.

2. Update the recovered domains for sensitive variables in the probed multiplier.

   (1) For probe types 1–2: if $i \notin I_x$, add i to I_x; if $i \notin I_y$, add i to I_y. For probe type 2, also add b_i to $\mathcal{R}$. Add x_i, y_i into $\mathcal{S}$.

   (2) For probe type 3: add z_{ij} to $\mathcal{R}$. If $i \notin I_x$, add i to I_x; if $j \notin I_y$, add j to I_y. If $x_j y_i + z_{ij} \in \mathcal{O}$ and $i \notin I_y$, add i to I_y. Add x_i, y_j into $\mathcal{S}$.

   (3) For probe type 4: add b_j and z_{ij} to $\mathcal{R}$. If $i \notin I_x$, add i to I_x. If $j \notin I_y$ and $y_j + b_j \in \mathcal{O}$, add j to I_y. If both $x_j b_i + z_{ij}$ and $y_i + b_i$ belong to $\mathcal{O}$ and $i \notin I_y$, add i to I_y. Add x_i into $\mathcal{S}$.

   (4) For probe type 5: add b_i to $\mathcal{R}$. If $i \notin I_y$, add i to I_y. Add y_i into $\mathcal{S}$.

[8]The observations add to $\mathcal{O}^t$ must be of these three forms, since we modified the definition of $\rho(q_i)$ for DOM-*dep* and DOM-*indep* to avoid the appearance of terms like $x_i y_i + L(x_i, y_i)$ and $x_i b_i + L(x_i, y_i)$.

(5) For probe type 6: add x_i and y_i to $\mathcal{S}$; add $(x_i y_k + z_{ik})_{i\neq k,\ 0\leq k\leq d}$ to $\mathcal{E}$. If $i \notin I_x$, add i to I_x; if $i \notin I_y$, add i to I_y. **Let** $K_1 = \{k \mid z_{ik} \in \mathcal{O},\ k \notin I_y,\ 0 \leq k \leq d\}$**. For all** $k \in K_1$**, add** k **to** I_y**.** Add q_i into $\mathcal{S}$.

(6) For probe type 7: add x_i and y_i to $\mathcal{S}$; add $(x_i b_k + z_{ik})_{i\neq k,\ 0\leq k\leq d}$ and $(y_k + b_k)_{i\neq k,\ 0\leq k\leq d}$ to $\mathcal{E}$; add b_i to $\mathcal{R}$. If $i \notin I_x$, add i to I_x; if $i \notin I_y$, add i to I_y. **Let** $K_1 = \{k \mid z_{ik} \in \mathcal{O},\ k \notin I_y,\ 0 \leq k \leq d\}$ **and** $K_2 = \{k \mid b_k \in \mathcal{O},\ k \notin I_y,\ 0 \leq k \leq d,\ k \neq i\}$**. For all** $k \in K_1 \cup K_2$**, add** k **to** I_y**.** Add q_i into $\mathcal{S}$.

(7) For probe types 8–9: add q_i to $\mathcal{S}$. If $i \notin I_q$, add i to I_q.

Let $\mathcal{P}_s$, $\mathcal{P}_e$, and $\mathcal{P}_r$ be the partition (shares, masked expressions, and masks) of $\mathcal{P}$, which is the observation set given by the $(t+1)$-th probe. The partition for each probe type is listed in Table 2.

Table 2: The partition of observations given by the $(t+1)$-th probe

Type	Shares ($\mathcal{P}_s$)	Masked Expressions ($\mathcal{P}_e$)	Masks ($\mathcal{P}_r$)
1	$\delta(x_i), \delta(y_i)$	$\mu(x_i), \mu(y_i)$	$\eta(x_i), \eta(y_i)$
2	$\delta(x_i), \delta(y_i)$	$\mu(x_i), \mu(y_i)$	$\eta(x_i), \eta(y_i), b_i$
3	$\delta(x_i), \delta(y_j)$	$\mu(x_i), \mu(y_j)$	$\eta(x_i), \eta(y_j), z_{ij}$
4	$\delta(x_i)$	$\mu(x_i)$	$\eta(x_i), b_j, z_{ij}$
5	$\delta(y_i)$	$\mu(y_i)$	$\eta(y_i), b_i$
6	x_i, y_i	$(x_i y_k + z_{ik})_{k\neq i}$	$\emptyset$
7	x_i, y_i	$(y_k + b_k)_{k\neq i}, (x_i b_k + z_{ik})_{k\neq i}$	b_i
8	q_i	$\emptyset$	$\emptyset$
9	q_i	$\emptyset$	$\emptyset$

There are only three ways for the adversary to recover domains of sensitive variables: (1) from observed shares; (2) by using known masks in $\mathcal{R}$ to unmask expressions in $\mathcal{E}$; (3) by identifying multiple masked expressions in $\mathcal{E}$ that share the same mask (the analysis for this case is similar to the ones in DOM-*dep* and DOM-*indep*, we omit this case hereafter).

Learn domains from observed shares. However, apart from the above three ways, there might be concerns that whether shares in $\mathcal{S}$ can be combined with observations in $\mathcal{E} \cup \mathcal{R}$ and other shares to recover domains of a sensitive variable. For example, consider the second-order NI multiplication $v = ab$ by Belaid *et al.* [BBP+16]. If v_0, b_1 is added to $\mathcal{O}^t$ by the second probe and the first probe is placed at r_0, then the adversary can recover all three domains by combining v_0 with b_1, r_0.

$$\begin{aligned} v_0 &= a_0 b_0 + r_0 + a_0 b_2 + a_2 b_0 \\ v_1 &= a_1 b_1 + r_1 + a_0 b_1 + a_1 b_0 \\ v_2 &= a_2 b_2 + r_0 + r_1 + a_0 b_2 + a_2 b_0 \end{aligned} \tag{1}$$

However, in our settings, when an output share v_i of a multiplier $\mathcal{M}$ which implements $v = ab + L(a, b)$ is added to $\mathcal{O}^t$, it can be viewed as a fresh mask to the existing internal observations of $\mathcal{M}$ in $\mathcal{O}^t$. This is because the multiplication we used in $\tilde{C}$ is SNI in standard probing model. When there are $t \leq d$ probes, the number of internal probes placed inside $\mathcal{M}$ is no more than t. By induction, the recovered domains of operands of $\mathcal{M}$ will be no more than t. As a result, v_i is strongly non-interfered with the internal observations of $\mathcal{M}$. So it can not be combined with $\mathcal{E} \cup \mathcal{R}$ and other shares to recover domains of sensitive variables that contribute to the computation of v_i.

On the other hand, a share x'_i in $\mathcal{S}$ can be combined with observations in $\mathcal{E} \cup \mathcal{R}$ to reconstruct output shares in domain i of a multiplier $\mathcal{M}'$ that uses x' as an input. However, the adversary can only perform this combination when they adversary places a probe which has access to the i-th output share of $\mathcal{M}'$. The recovered domains through this case is covered by the statement in gray in step 2. The adversary has not direct access to the shares x_i, y_i or y_j through the probe. However, they can reconstruct x_i, y_i or y_j through the observations added to $\mathcal{O}$ in step 1. Since we add these inferred shares in $\mathcal{S}$ in step

2, we can view this case as a variant of the case where the adversary learn domains from shares in $\mathcal{S}$.

If the adversary recover domains from shares, we should ensure he has only add one domain to each variable through this process. For all probe types but probe type 3, he either add or does not add i to I_u for any bit $u \in \pi(x)$ or $u \in \pi(y)$, or $u \in \pi(q)$. Therefore, in this case, the number of recovered domains through $\mathcal{P}_s$ is increased by at most one.

However, for probe type 3, the adversary might add domain i for $u \in \pi(x)$, and add domain j for $u \in \pi(y)$. If $\pi(x) \cap \pi(y) \neq \emptyset$, then the adversary can add two different domains to the same variable. This possibility is prevented by the condition $\pi(x) \cap \pi(y) = \emptyset$ in Proposition 4. Thus the number of recovered domain through $\mathcal{P}_s$ is also increased by at most one in this case.

Combining $\mathcal{P}$ and $\mathcal{E}$. As in the proof for Proposition 1, we only need to consider the cases combining $\mathcal{P}_r$ and $\mathcal{E}^t$ or combining $\mathcal{P}_e$ and $\mathcal{R}^t$. The other cases can be excluded through the same analysis in the proof of Proposition 1.

The adversary can combine $\mathcal{P}_r$ and $\mathcal{E}^t$. From Table 2, we observe that the masks in $\mathcal{P}_r$ are either from a $\eta(\cdot)$ function or from the randomness used directed by the probed masked computation, i.e., z_{ij}, b_i, b_j. For the former, $\eta(x_i), \eta(y_i)$ or $\eta(y_j)$ is not empty only if at least one bit u in x_i, y_i, or y_j is from an output bits of DOM-*dep*. Thus the $\eta(\cdot)$ function will contain b'_i from the corresponding DOM-*dep*. However, b'_i can only be used to unmask $y'_i + b'_i$ to obtain y'_i, while domain i has already been added to $I_{y'}$ through the shares in $\mathcal{S}$ since $y' \in \pi(u) \subseteq \pi(x)$ or $y' \in \pi(u) \subseteq \pi(y)$. As a result, we only need to consider the masks z_{ij}, b_i or b_j.

For $z_{ij} \in \mathcal{P}_r$ in probe types 3 and 4, the analysis is similar to the analysis of the probe group (3) in Process II and probe group (4) in Process I. The domain recovery caused by combining z_{ij} and $\mathcal{E}^t$ is a aligned domain recovered. It is done in step 2(5) and step 2(6) in the algorithm run by the adversary in this proof.

For $b_i \in \mathcal{P}_r$ in probe types 2, 5 and 7, the analysis is similar to the analysis of the probe group (1), (3) and (4) in Process I. We omit it here.

For $b_j \in \mathcal{P}_r$ in probe type 4, the adversary can possibly recover domain j of y. Note that the adversary add domain i to variables in $\pi(x)$ through this probe. If $y \cap \pi(x) \neq \emptyset$, then the adversary can add two different domains i and j to the same variable in $y \cap \pi(x)$. This possibility is prevented by the condition $\pi(x) \cap y = \emptyset$ in Proposition 4.

Therefor, after considering the shares $\mathcal{P}_s$ and the combination of $\mathcal{P}_r$ and $\mathcal{E}^t$, the number of recovered domain from $\mathcal{P}_s$ is also increased by at most one for all probes.

The adversary can combine $\mathcal{P}_e$ and $\mathcal{R}^t$. This step is done through step 1(2) and step 1(3) in the algorithm in this proof. Such combinations only aligns the domains of the two operands of a multiplication. Since we have ensure that the number of recovered domains of each sensitive variable through $t+1$ probes is no more than $t+1$, the recovered domains for all variable are still no more than $t+1$ after alignment.

Finally, all cases are covered, we can conclude that the induction holds and thus Proposition 4 is true.

□

From the proof of Proposition 4, we know that what make DOM-*dep* and DOM-*indep* not trivially composable is that domain i and j of the same variable can be recovered with one probe through direct domain recovery if we do not pose constraints on the inputs. We give two attacks in Appendix A to demonstrate this.

Proposition 5. *If several DOM-dep or DOM-dep-SNI multipliers in the masked circuit $\tilde{C}$ in Proposition 4 share the same operand y in the same cycle, they can share the registers $(y_i + b_i)_{0 \leq i \leq d}$ and share the randomness $(b_i)_{0 \leq i \leq d}$ used to multiply the other operand x.*

Proposition 5 is motivated by the randomness reuse strategy introduced in [HB25, Theorem 2,3]. Like the blinding randomness used in the HPC3s, $(b_i)_{0 \leq i \leq d}$ used in DOM-

dep does not occur in the expression of output shares of DOM-*dep*, making it suitable for randomness reuse.

Proof. (Sketch.) The proof for Proposition 5 and Proposition 4 is very similar. The algorithm to record observations and recovered domains for Proposition 5 is identical to the one in proof for Proposition 4.

The difference between proofs for Proposition 5 and Proposition 4 is that in induction step, we should carefully consider the influence of reusing randomness on domain recovery.

We can observe that with or without randomness reuse, b_i can only be combined with terms like $b_i + y_i$ to recover y_i. The access to b_i will not affect the domain recovery of other sensitive variables. Thus it suffices to ensure that, in each step of the algorithm for Proposition 5, if the process of recovering shares of y involves the shares of b, the number of recovered domains of y stays the same as when randomness is not reused.

Firstly, the blinding randomness involved in step 1 and step 2 of the algorithm is not shared since they are not used in the same cycle. Thus, we can discuss them separately without considering that b appears in both step 1 and step 2.

Step 1(2), 2(1), 2(3), 2(4), and 2(6) involves the shares of b in recovering domains of y.

For step 1(2) and 2(6), whether $b_k \in K_2$ is reused or not would not affect the analysis for the number of previously placed probes. Although the value of $|K_1| + |K_2|$ might change, the expression for the number is still $|K_1|+|K_2|$. Thus the aligned domain recovery property preserves when randomness is reused.

In step 2(1), 2(4), y_i and b_i is added to $\mathcal{O}$ at the same time. Thus y_i is recovered without the help of b_i, and the recovered domain of y stays the same as the case when randomness is not reused.

For step 2(3), whether b_i or b_j is reused do not change what the adversary do, the adversary still add at most one domain for y.

Therefore, in all cases, the reused randomness does not change what the adversary do and does not affect the number of recovered domains for a common operand y.

□

Proposition 6. *If the output bits of the masked circuit $\tilde{C}$ in Proposition 4 are all from DOM multipliers, and a full-rank square matrix transformation in $\mathbb{F}_2$ is applied on the output bits in each share domain and then the results of linear transformation are stored in registers, the whole circuit composed of $\tilde{C}$, the linear transformation and the output registers are d-MO-SNI.*

The proof for this proposition is similar to the proof of Proposition 3, thus we omit it here but provide it in Appendix B.2.

The drawback of MO-SNI is that an MO-SNI gadget cannot have a linear layer at its input without refreshing some of these inputs. However, Proposition 6 shows that after full-rank linear transformation on the outputs of a MO-SNI gadget, the gadget is still MO-SNI. This property will enable us the design composable masked implementations for some specific cryptographic functions. For example, for an iterative function $G = F \circ F \circ F$ with $F = L_2 \circ N \circ L_1$ where L_1, L_2 are the linear layers and N the non-linear layer. We can first design a MO-SNI implementation of N, and thus the implementation of the function $F' = L_1 \circ L_2 \circ N$ is also MO-SNI. The implementation of G can be transformed to $L_1^{-1} \circ F' \circ F' \circ F' \circ L_1$. Then the implementation of G is composed of a input linear layer L_1 and a MO-SNI non-linear component F' and an output linear layer L_1^{-1}, which is easier to give security proofs on arbitrary protection orders.

4 Masked AES implementations with arbitrary protection order

4.1 Masked $\mathbb{F}_{2^8}$ Inverters

Figure 5 and Figure 6 show our masked Inverter for Canright's and Hadzic's decomposition, respectively. There are 6 and 8 communicative multiplication functions in Canright's and Hadzic's decomposition. We should decide whether DOM-*dep* or DOM-*indep* is used to masked these multiplication functions.

In Figure 5, x, p are the four LSBs of input and output of inverter in $\mathbb{F}_{2^8}$. a, c are the two LSBs of input and output of inverter in $\mathbb{F}_{2^4}$. In Figure 6, x, y are the four LSBs and MSBs of the input of the inverter. The output components p, q, r, and s represent the two LSBs up to the two MSBs.

We assume the input to the inverter is stored in registers. We start with Canright's decomposition.

(1) For the multiplier A in stage 1, since $\pi(x) = \{x\}$ and $\pi(y) = \{y\}$, $\pi(x) \cap \pi(y) = \emptyset$, this multiplication can be masked by DOM-*indep*.

(2) For the multiplier B in stage 2, since $\pi(a) = \{x, y\}$, $\pi(b) = \{x, y\}$, $\pi(a) \cap \pi(b) \neq \emptyset$, this multiplication can not be masked by DOM-*indep*. However, it can be masked by DOM-*dep* since $\pi(a) \cap b = \emptyset$. Moreover, it does not matter whether a or b is connected $\boldsymbol{y}$ port since $\pi(a) \cap b$ and $\pi(b) \cap a$ are both empty set.

(3) For the multiplier C_1 in stage 3, $\pi(a) = a, \pi(u) = \{a, b\}$, the intersection of $\pi(a)$ and $\pi(b)$ is not empty set. Therefore, it can not be masked by DOM-*indep*. However, the intersection for $\pi(a)$ and u is empty set, it can be masked by DOM-*dep* in the condition that the sharing of u is connected to the $\boldsymbol{y}$ port of DOM-*dep*. On the other hand, $\pi(u) \cap a = a$, if a is connected to the $\boldsymbol{y}$ port, the implementation has a $(\lceil \frac{d}{2} \rceil + 1)$-order flaw.

(4) The analysis for multiplier C_2 in stage 2 is the same as C_1. As a result, u must be connected to the $\boldsymbol{y}$ port of C_2. Since C_1 and C_2 share the same operand u which is connected to their $\boldsymbol{y}$ port, they can reuse the blinding randomness.

(5) For multiplier D_1 in stage 4, $\pi(v) = \pi(c) \cup \pi(d) = \{a, u, b\}$. Since $\pi(x) \cap \pi(u) = \emptyset$, this multiplication can be masked by DOM-*indep*. The same analysis holds for D_2.

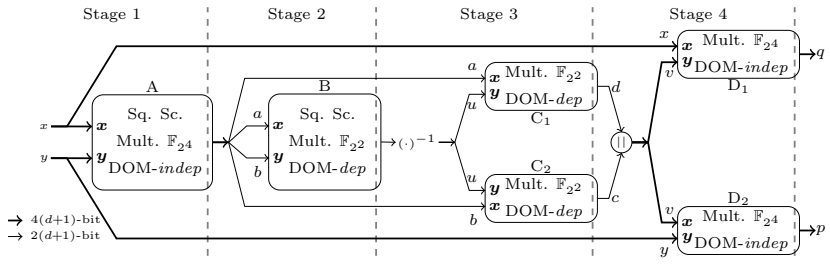

Figure 5: Masked Canright's $\mathbb{F}_{2^8}$ Inverter

For the implementation of Hadzic's inverter, we have the following analysis.

(1) The analysis for the multiplier A and multiplier B_2 is the same as the analysis for multiplier A and B in Canright's inverter.

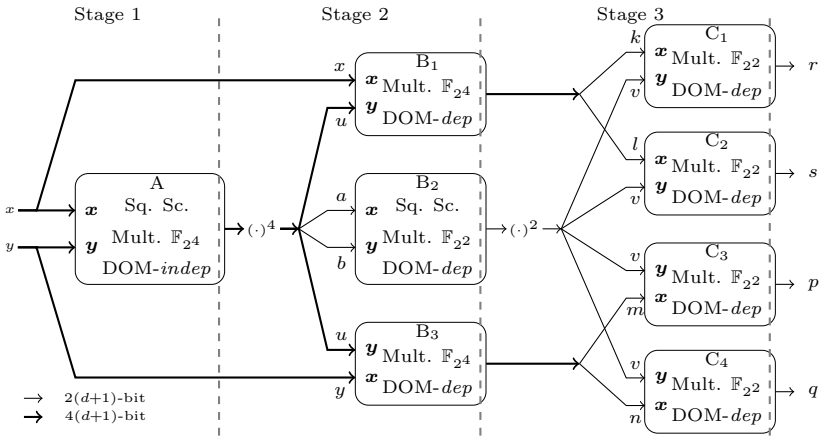

Figure 6: Masked Hadzic's $\mathbb{F}_{2^8}$ Inverter

(2) The analysis for the multiplier B_1 and B_3 is the same as analysis for the multiplier C_1 and C_2. u must be connected to the $\boldsymbol{y}$ port for both multipliers and thus B_1 and B_3 can reuse the blinding randomness.

(3) The analysis for multiplier C_1 to C_4 in stage 3 is the same as B_2 in stage 2. They must be masked by DOM-*dep* but it does not matter which variable is connected to the $\boldsymbol{y}$ port. In order to reuse randomness, we connect v to the $\boldsymbol{y}$ port of the four multipliers and thus the four multipliers can share the blinding randomness.

4.1.1 Lower order optimization

Cassiers *et al.* [CGM+24] and Hadzic *et al.* [HB25] have provided efficient composable implementations of AES S-Boxes using HPC3.1 (HPC3o) and/or HPC1 recently.

When implementing our designs, we compared their implementations to ours and find that the use of DOM-*dep* at small orders causes disadvantages in terms of area. Therefore, we give optimizations on small orders. Before illustrating our optimization, We would like to show the comparison first.

For first-order implementations, their smallest designs for first-order are exclusively composed of HPC3o or HPC3.1. Since the implementation costs for the first order implementation of HPC3.1, HPC3o, and optimized DOM-*dep* are almost the same, our first-order implementation are comparable to theirs.

For higher-order implementations, we use the regular DOM-*dep* multipliers. Table 3 shows required randomness, number of multipliers, and number of registers required by DOM multipliers and the HPC3s when security order $d \geq 2$. We observe complexity of amount of required randomness and registers of DOM-*dep* is asymptotically smaller than the variants of HPC3. However, the advantage of DOM-*dep* in terms of randomness and register usage only becomes apparent at orders higher than 2. (1) Although the closed-form expressions for the number of random bits required by DOM-*dep* and HPC3.1 (HPC3o) differ with respect to the security order d, their values coincide at $d = 2$. This makes DOM-*dep* has no advantage in terms of randomness. (2) Secondly, the area for registers are neck-to-neck between the gadgets when $d = 2$. (3) Finally, in terms of required number for multipliers, DOM-*dep* uses one more unmasked multiplier than HPC3.1.

When the field is large and the protection order is small, the area disadvantage caused by the additional multiplier can not be made up by other factors. This can be illustrated by comparison between our second-order implementation and the one in [HB25] for Hadzic's decomposition. The area of our implementation without RNG is much larger than the one

in [HB25]. This is because our implementation uses two more unmasked multiplication in $\mathbb{F}_{2^4}$ for the multiplier B_1 and B_3 in Figure 6. The area for the two multipliers is about 500 GE.

However, this disadvantage does not appear in the masking of Canright's decomposition. This is because although we use less efficient DOM-*dep* to implement multiplier B, C_1, C_2, we use more efficient DOM-*indep* to implement A, D_1, D_2. The field where DOM-*dep* is used is relatively small thus the area for the additional multipliers is smaller.

This motivates us to find an alternative method to avoid the use of DOM-*dep* when masking at lower order, thus decreasing the area of our implementations. Here, we take the advantage of SNI multiplications with one-cycle latency introduced in [MPZ22]. These low-latency SNI multiplications required $2(d+1)^2$ random bits for security order d. However, it has optimizations when $d \leq 4$. We opted to use these optimization for first- and second-order.

Table 3: Comparison to State-of-the-art HPC3 Gadgets when $d \geq 2$

Gadget	Randomness	# Multipliers	# registers
DOM-*indep*	$\frac{d(d+1)}{2}$	$(d+1)^2$	$(d+1)^2$
DOM-*dep*	$\frac{d(d+1)}{2} + d + 1$	$(d+1)^2 + 1$	$(d+1)^2 + 2(d+1)$
HPC3.1 [HB25]	$d(d+1)$	$(d+1)^2$	$2(d+1)^2$
HPC3.2 [HB25]	$d(d+1)$	$(d+1)^2$	$2(d+1)^2 - (d+1)$
HPC3o [CGM+24]	$d(d+1)$	$2(d+1)^2 - (d+1)$	$2(d+1)^2 - (d+1)$
HPC3 [KM22]	$d(d+1)$	$2(d+1)^2$	$2(d+1)^2$

Specifically, we replace the multipliers A and B with a 1-cycle SNI multiplier for Canright's inverter, thus multiplier C_1 and C_2 can be replaced by DOM-*indep*. Similarly, we replace the multipliers A and B_2 with a 1-cycle SNI multiplier for Hadzic's inverter, thus the rest multipliers can all be replaced with DOM-*indep*.

4.2 Round-based AES implementations

4.2.1 Architecture

We adopt the round-based architecture of masked AES implementations by Cassiers *et al.* [CGM+24]. In this architecture, the latency of S-Box can be modified by a parameter. As a result, we can simply replace their S-Box with ours and change the latency accordingly. However, using their implementation directly will result a latency of 5-cycle and 4-cycle for Canright's S-Box and Hadzic's S-Box, respectively. This is not optimal because the result of input linear mapping must be stored in registers, consuming one clock cycle. Since the results of each round require a register stage, we can merge the two register stages into one register stage. Gigerl *et al.* have used this method to design a second-order AES implementation [GKM+24].

Specifically, we store the results of $L(p_i^j \oplus k_i^j)$ to the registers that is used to store p_i^j for $1 \leq j \leq 16$, where p_i^j and k_i^j are the j-th byte of plaintext state and key state in the i-th round, L denotes for the inputs linear mapping of Canright's S-Box. For key scheduling, we store the results of $L(k_i^j)$ to the registers that is used to store k_i^j for $13 \leq j \leq 16$. With these modifications, when key column $(k_i^j)_{13 \leq j \leq 16}$ is routed to SubWord or plain state bytes are routed to SubBytes, input linear mapping is already done and the results is stored in registers, which satisfies our assumption for masking $\mathbb{F}_{2^8}$ inverter.

4.2.2 Security of our round-based AES implementations

Initially, our AES core loads key state and plaintext state to registers follows the description in last section. After the key and plaintext state registers are initialized, the AES core starts a 10-round encryption as follows: (1) inversion of $\mathbb{F}_{2^8}$ for each byte (shift rows is implemented through routing the bytes to the masked $\mathbb{F}_{2^8}$ inverter); (2) output linear

mapping for each byte; (3) mixing column for each column (bypassed in round 10); (4) adding key byte from the key registers for each byte; (5) input linear mapping for each byte (bypassed in round 10) and storing the round results;

With these modifications, both the new round function and the SubWord are d-MO-SNI after the results is stored in registers since the both output mapping and MixColumns are full-rank square matrix transformation. Although in Proposition 6, we did not mention that a linear term (the addition of round key in our new AES round function) can be incorporated in the middle of the matrix transformation, it does affect our conclusion, since the linear terms just make a contribution in the sensitive intermediate variable q' in our proof for Proposition 6. It does not affect the randomness used in this gadgets, which is the key to prove MO-SNI. As the result, our design is d-NI when we do not consider the initial input mapping.

If the adversary places $t_1 \leq d$ probes in the initial input mapping before starting the new round function, he learns at most $t_1 \leq d$ bit input shares about the rest of the encryption (or at most $t_1 \leq d$ domains of the initial sensitive variables[9]). When he combines with $t_2 \leq d - t_1$ probes from the rest of the encryption, he still learns at most d shares of the input sensitive variables to the rest of the encryption. Thus we conclude our masked AES is secure at arbitrary order in glitch-extended probing model.

On the other hand, after the i-th round results is routed to the S-Box module, we set the value of registers that stores the round results to zero, thus when $(i + 1)$-th round results is written back, there is no transitional leakage.

As a side note, our proof techniques can be used prove the security of the masked AES implementation provided by Gross *et al.* [GMK17].

5 Comparison and Security Evaluations

In this section, we present implementation costs for our masked AES S-Box and round-based AES implementations[10], and the security evaluation results for our masked AES-SBox without input linear mapping.

5.1 Comparison to State-of-the-art d-PINI AES S-Box Implementations

In term of implementation costs, we followed the synthesis flow of [HB25]. We use Yosys [WGK13] to elaborate, optimize and synthesize our implementations. Then we use ABC [G+07] to technology-map the circuits with NanGate45 as the target library. The area overhead required by Trivium RNG is 39.4 *gate equivalent* (GE) per random bit [CMM+24, CGM+24, HB25].

The implementation costs of our masked AES S-Box and other state-of-the-art AES S-Box implementations are shown in Table 4 (The latency of the designs is listed in the column l).

Many recent work used the d-PINI framework to implement AES S-Box with arbitrary protection order [KMMS22, MCS22, KM22, WFP+24, CGM+24, HB25]. However, the earliest ones [KMMS22, MCS22, KM22, WFP+24] are masked at bit-level, which can not utilize the advantage of field-level masking, as explained in [HB25]. Cassiers *et al.* [CGM+24] and Hadzic *et al.* [HB25] proposed the gadgets HPC3o and HPC3.1, which aim at masking at field-level, reducing the randomness and area overhead significantly.

However, as demonstrated by this work, masked $\mathbb{F}_{2^8}$ can be secure at arbitrary order without fulfilling the d-PINI property, which further decrease the implementation cost of higher-order hardware masking. We can observe from the table, given the same latency,

[9] Note that the t_1 domains of the initial sensitive variables can only be used to infer t_1 bit input shares for the rest of the encryption.

[10] Available at MaskedAES.

only the 3-cycle S-Box designs provided by Hadzic *et al.* [HB25] at order 3 and 4 is slightly smaller than us in terms of area without RNG. However, when the area of RNG is included, our implementations save 9.2% and 11% of the area at order 3 and 4.

In other cases, we have significant improvements on area and randomness to the implementations of the same latency.

Table 4 lists implementation costs of our design and state-of-the-art d-PINI AES S-Box.

In our masked AES designs, the linear mapping is performed outside the SubBytes module. We have included it without its results storing in registers in synthesis of area for a fair comparison.

Table 5 lists the number of random bits required by our masked inverter designs and some representative designs in literature with security order d.

The DOM design proposed in [GMK17] has the least randomness requirements with arbitrary protection order. However, it has a latency of 6 clock cycles. Our design for Canright's inverter achieves a latency of only 4 clock cycles at the cost of additional $4(d[+1])$ random bits for security order d. The design in [CGM+24] also has a latency of 4 clock cycles. Since it uses HPC3o and HPC1, it has larger demands on randomness than ours.

As far as we know, the 3-cycle design in [HB25] is the only design which achieves the latency of 3 clock cycles for $\mathbb{F}_2^8$ inverter. However, since it uses HPC3.1 as the underlying gadgets, it has a large demand on the randomness despite using the randomness reuse strategy. According to Proposition 4, we can design a secure 5-cycle masked $\mathbb{F}_2^8$ inverter with only DOM-*indep* and DOM-*indep*-SNI. To achieves this goal, we only need to insert pipeline stages after stage 1 and stage 2 in Figure 6, and replace all DOM-*dep* in Figure 6 with DOM-*indep*. This design will only need $11d(d+1)$ random bit to achieve d-th order security. In the 3-cycle design proposed by this paper, we only require additional $6(d[+1])$ random bits to reduce the latency to 3 clock cycles where $2(d[+1])$ random bits are shared by multipliers C1, C2, C3 and C4, and the other $6(d[+1])$ random bits are shared by B1, B2 and B3 in Figure 6.

5.2 Comparison to Round-Based Masked AES Implementations

Table 6 lists the implementations cost of our round-based masked AES implementations and the ones in [CGM+24] and [GKM+24]. There are also other round-based implementations that is secure at arbitrary order [KM22, KMMS22, MCS22]. However, they are less efficient than the round-based implementations in [CGM+24] so we do not list them here. Please refer to [CGM+24] for a comparison between these implementations.

The area for PRNG is not estimated by 39.4 GE per random bit like the case of AES S-Box. Instead, we have PRNG included in our synthesis. Although our implementations based on Canright's inverter consumes less random bit than the one based on Hadzic's, the number of total Trivium RNG instances used to generate random bits in both designs is the same for first-order. Therefore, the area of first-order implementation based on Canright's decomposition is slightly higher than the one based on Hadzic's decomposition since the standalone area of first-order Hadzic's inverter is less than the one based on Canright's inverter.

Compared to the masked AES implementations provided by Cassiers *et al.* [CGM+24], our implementations save at least 13% in terms of area (PRNG included) when security order $d \leq 4$, even the implementations with 19.6% less latency. Note that the latency of 41 clock cycles can be also archived by using S-Box designs in [HB25], which does not provide masked AES implementations. However, we believe the area of the round-based implementations using their S-Boxes will be higher than ours since their S-Boxes consume more area and significantly more randomness than ours in most of the cases.

Nevertheless, our second-order implementation is less efficient than the one provided by Gigerl *et al.* [GKM+24], since they use the technique *changing of the guards* to save

Table 4: Area and randomness cost of our masked AES S-Boxes compared to state-of-the-art d-PINI designs, annotated with percentages showing how much cost is saved by the Hadzic design (black) and the Canright design (gray) with optimizations on first- and second-order.

AES S-Box	l	d	Random bits		Area (GE) standalone		Area (GE) with PRNG	
This work Hadz. repr. $d = 1, 2$ opti.	3	1	28	0.0% 14.3%	1642	0.0% −0.9%	2745	0.0% 5.2%
		2	84	0.0% 14.3%	3896	0.0% 3.3%	7205	0.0% 8.3%
		3	156	0.0% 20.5%	7696	0.0% 17.8%	13842	0.0% 19.0%
		4	250	0.0% 20.0%	11574	0.0% 17.3%	21424	0.0% 18.5%
This work Canr. repr. $d = 1, 2$ opti.	4	1	24	−16.7% 0.0%	1657	0.9% 0.0%	2602	−5.5% 0.0%
		2	72	−16.7% 0.0%	3767	−3.4% 0.0%	6604	−9.1% 0.0%
		3	124	−25.8% 0.0%	6328	−21.6% 0.0%	11214	−23.4% 0.0%
		4	200	−25.0% 0.0%	9574	−20.9% 0.0%	17454	−22.7% 0.0%
Hadz. repr. no opti.	3	1	28	0.0% 14.3%	1840	10.8% 9.9%	2943	6.7% 11.6%
		2	84	0.0% 14.3%	4664	16.5% 19.2%	7973	9.6% 17.2%
Canr. repr. no opti.	4	1	22	−27.3% −9.1%	1869	12.2% 11.4%	2736	−0.3% 4.9%
		2	66	−27.3% −9.1%	3805	−2.4% 1.0%	6406	−12.5% −3.1%
HB25 [HB25] Hadz. repr.[1]	3	1	32	12.5% 25.0%	1875	12.4% 11.6%	3136	12.5% 17.0%
		2	96	12.5% 25.0%	4176	6.7% 9.8%	7958	9.5% 17.0%
		3	192	18.8% 35.4%	7687	−0.1% 17.7%	15252	9.2% 26.5%
		4	320	21.9% 37.5%	11465	−1.0% 16.5%	24073	11.0% 27.5%
HB25 [HB25] Canr. repr.[1]	4	1	29	3.4% 17.2%	1806	9.1% 8.2%	2948	6.9% 11.7%
		2	96	12.5% 25.0%	3880	−0.4% 2.9%	7662	6.0% 13.8%
		3	192	18.8% 35.4%	6563	−17.3% 3.6%	14127	2.0% 20.6%
		4	300	16.7% 33.3%	9893	−17.0% 3.2%	21713	1.3% 19.6%
Compress [CGM+24] HPC3o+HPC1 Canr. repr.[1]	4	1	36	22.2% 33.3%	1950	15.8% 15.0%	3368	18.5% 22.7%
		2	96	12.5% 25.0%	4560	14.6% 17.4%	8342	13.6% 20.8%
		3	192	18.8% 35.4%	8060	4.5% 21.5%	15624	11.4% 28.2%
		4	300	16.7% 33.3%	12480	7.3% 23.3%	24300	11.8% 28.2%
Low-Latency HPC[KM22] HPC3[3]	4	1	68	58.8% 64.7%	1849	11.2% 10.4%	4528	39.4% 42.5%
		2	204	58.8% 64.7%	4855	19.8% 22.4%	12892	44.1% 48.8%
		3	408	61.8% 69.6%	9261	16.9% 31.7%	25336	45.4% 55.7%
AGMNC [WFP+24] AND-XOR1[3]	6	1	66	57.6% 63.6%	2895	43.3% 42.8%	5495	50.0% 52.6%
		2	165	49.1% 56.4%	5745	32.2% 34.4%	12246	41.2% 46.1%
		3	330	52.7% 62.4%	9243	16.7% 31.5%	22245	37.8% 49.6%
		4	495	49.5% 59.6%	13314	13.1% 28.1%	32817	34.7% 46.8%
AGMNC [WFP+24] AND-XOR2[3]	6	1	33	15.2% 27.3%	3967	58.6% 58.2%	5267	47.9% 50.6%
		2	99	15.2% 27.3%	9078	57.1% 58.5%	12978	44.5% 49.1%
		3	198	21.2% 37.4%	16239	52.6% 61.0%	24040	42.4% 53.4%
		4	330	24.2% 39.4%	25469	54.6% 62.4%	38471	44.3% 54.6%
Handcraf. [MCS22] HPC2[2]	6	1	34	17.6% 29.4%	3213	48.9% 48.4%	4552	39.7% 42.8%
		2	102	17.6% 29.4%	6705	41.9% 43.8%	10723	32.8% 38.4%
		3	204	23.5% 39.2%	11515	33.2% 45.0%	19552	29.2% 42.6%
AGEMA [KMMS22] HPC1 Optimized Pipelined[1]	8	1	68	58.8% 64.7%	4263	61.5% 61.1%	6942	60.5% 62.5%
		2	170	50.6% 57.6%	7839	50.3% 51.9%	14537	50.4% 54.6%
		3	340	54.1% 63.5%	12085	36.3% 47.6%	25481	45.7% 56.0%
		4	510	51.0% 60.8%	16919	31.6% 43.4%	37013	42.1% 52.8%
AGEMA [KMMS22] HPC2 Optimized Pipelined[1]	8	1	34	17.6% 29.4%	5339	69.2% 69.0%	6678	58.9% 61.0%
		2	102	17.6% 29.4%	11205	65.2% 66.4%	15223	52.7% 56.6%
		3	204	23.5% 39.2%	19217	60.0% 67.1%	27254	49.2% 58.9%
		4	340	26.5% 41.2%	29267	60.5% 67.3%	42663	49.8% 59.1%

[1] Synthesized with Yosys to the NanGate45 design kit.
[2] Synthesized with Cadence to the TSMC-N65 design kit.
[3] Synthesized with Synopsys to the NanGate45 design kit.

Table 5: Randomness comparison with other designs

$\mathbb{F}_2^8$ inverter	Latency	Random bits
This work	3	$11d(d+1) + 6(d[+1])^*$
	4	$9d(d+1) + 4(d[+1])^*$
HB25 [HB25]	3	$16d(d+1)$
Compress [CGM+24]	4	$12d(d+1) + 12r(d)^\dagger$
DOM [GMK17]	6	$9d(d+1)$

* $+1$ in $[\cdot]$ is only needed for $d > 1$.
† $r(d) = [1, 2, 4, 5, 7, 9, 11, 13, 15, 17]$ for $1 \leq d \leq 10$.

randomness. As a result, their implementation requires only 3200 random bits in one encryption while we need 14400 and 16800 random bits per encryption for our second-order implementations using Canright's inverter and Hadzic's inverter, respectively.

Table 6: Area cost (RNG included) of our masked AES compared to state-of-the-art round-based AES designs, annotated with percentages showing how much cost is saved by the implementations based on Hadzic' decomposition (black) and Canright's decomposition (gray).

Design	Latency [cycles]	order	Area with PRNG [kGE]	
This work Hadz. repr. $d = 1, 2$ opti.	41	1	69.5	0.0% −0.4%
		2	163.7	0.0% 3.5%
		3	298.0	0.0% 14.8%
		4	445.8	0.0% 14.2%
This work Canr. repr. $d = 1, 2$ opti.	51	1	69.8	0.4% 0.0%
		2	158.0	−3.6% 0.0%
		3	253.9	−17.4% 0.0%
		4	382.7	−16.5% 0.0%
Compress [CGM+24] HPC3o+HPC1 Canr. repr.[1]	51	1	85.0	18.2% 17.9%
		2	190.9	14.3% 17.3%
		3	343.6	13.3% 26.1%
		4	521.5	14.5% 26.6%
DOM + COTG[2][GKM+24]	51	2	116.6	−40.4% −35.5%

[1] Synthesized with Yosys to the NanGate45 design kit.
[2] Synthesized with Cadence to the UMC-65 design kit.

5.3 Formal verification of our masked $\mathbb{F}_{2^8}$ inverter

Our designs for AES S-Box without the input mapping (the composition of $\mathbb{F}_{2^8}$ inverter and the output linear mapping) is d-NI[11] if the outputs are not stored in registers and is d-MO-SNI if the outputs are stored in registers. maskVerif [BBC+19] could verify d-NI and d-MO-SNI at the same time. Therefore, we use maskVerif to formally verify our masked implementations. All our implementations listed in Table 4 without the input linear mapping passed the d-NI verification if the outputs are not stored in registers. After that, we adapted our code to store the outputs to registers and repeat the verification steps. maskVerif is able to verify d-MO-SNI property of our masked implementations up to second-order. However, when verifying 3-MO-SNI for our masked implementations, maskVerif suffered from very slow performance, owing to the substantial complexity of

[11] However, if the input linear mapping is included, the design is not d-NI.

the verification process. For example, for a four-shared $\mathbb{F}_{2^8}$ inverter. The complexity for verifying $t_1 = 3, t_2 = 0$ with t_1 the number of external probes and t_2 the number of internal probes would be $\binom{4}{3}^8 = 65536$. This requires maskVerif about 20 days to verify. The number of internal observations is about 300 for our third order implementation using Canright's inverter. When verifying the case of $t_1 = 2, t_2 = 1$, the number of observation sets to be verified will be $\binom{4}{2}^8 \times 300$, about 500 millions. We once have maskVerif ran for 22 days, but it only verified the case $t_1 = 3, t_2 = 0$ and 4.5 million tuples out of 500 millions in the case of $t_1 = 2, t_2 = 1$. The program was terminated during a crash of the computer. We believe it would require at least two month for maskVerif to finish the verification. We do not have spare machine to run a program for such a long time (without rebooting the computer during the verification process). As far as we know, other verification tools are not able to verify MO-SNI. Therefore we give up the verification of d-MO-SNI property of our implementations for $d \geq 3$.

6 Conclusion

In this work, we revisited Domain-Oriented Masking. We have done a comprehensive study on the security analysis of DOM-*indep* and DOM-*dep*, including the security of individual DOM multipliers and compositional security of masked circuits composed of exclusively DOM multipliers. As a result, we identify a sufficient condition to build a masked circuit with arbitrary protection order using exclusively DOM-*indep* and DOM-*dep*. Finally, we are able to provide masked AES implementations with arbitrary protection order. Although these implementations do not fulfill d-PINI property, they are indeed secure at every order with significantly lower implementation costs.

Future directions include extending our results on arbitrary circuits and developing automated tools for generating masked circuits using the composition rule of DOM-*dep* and DOM-*indep*, and their SNI variants.

A Giving Attacks Based on Our Analysis

A.1 Revisiting the attack given by Moos *et al.*

DOM-*dep* is claimed to be able to deal with identical input sharing. However, this is not true as shown by Moos *et al.* [MMSS19]. Here, we revisit this attack.

Obviously, we should focus on the probes that can reveal one share domain of x and another share domain of y. Since the input sharing is identical for x and y, such probes is able to recover two share domains of the same variable per probe. Thus $\lceil \frac{d}{2} \rceil + 1$ probes is enough to recover x (one is need to placed on an output share).

Suppose the adversary places one probe on the output share q_i, then he will be able to recover $2d + 1$ field elements at the same time, i.e., x_i, $y_j + b_j$, t_{ij} for $0 \leq j \leq d$ and $j \neq i$.

Next, a better strategy for the adversary would be place probes on the term $t_{k_1 l_1}, \cdots, t_{k_{d_2} l_{d_2}}$ with $k_1, \cdots, k_{d_2}, l_1, \cdots, l_{d_2}$ different from each other and none of them equals to i. Otherwise, if the adversary places probes on t_{ij} or t_{ji}, one additional probe only provides information about one domain rather than two domains as analyzed in the following.

- If the adversary places a probe at $\rho(t_{ij}) = \{x_i, b_j, z_{ij}\}$, they would only get additional information about one share of y, i.e., y_j (combining b_j and $y_j + b_j$).

- If the adversary places a probe at $\rho(t_{ji}) = \{x_j, b_i, z_{ij}\}$, they would only get additional information about one share of x, i.e., x_j (since y_i is already known, the combination of $y_i + b_i$ and b_i reveals no more information).

Finally, let $i = 0, k_j = j, l_j = j + \lceil \frac{d}{2} \rceil$ with $1 \leq j \leq \lceil \frac{d}{2} \rceil$, then the attacker can launch the $(\lceil \frac{d}{2} \rceil + 1)$-th order attack for a d-order implementation of DOM-*dep* in [MMSS19].

A.2 A $(\lceil \frac{d}{2} \rceil + 1)$-th order flaw of the 5 staged AES S-Box implementation in [GMK16b]

Two variants of AES S-Box implementations is provided in [GMK16b]. One only uses the DOM-*indep* multipliers, which is 8 staged, and the other uses both DOM-*dep* and DOM-*indep*, which is 5 staged. We now use the results of our analysis to exhibit $(\lceil \frac{d}{2} \rceil + 1)$-th order attack of the 5-staged implementation for $d \geq 2$.

The leakage occurs in the first stage of the S-Box, where a DOM-*dep* takes the sahres of four bit MSB (m, n, p, q) and four bit LSB (r, s, t, u) of the results of a linear transformation (given in Equation 2 where h, v is LSB) as input $\boldsymbol{x}, \boldsymbol{y}$. Since the result of the linear transformation is not stored in registers, we have $\rho(x_i) = \{a_i, b_i, c_i, d_i, f_i, g_i, h_i\}$. Since $s_i = f_i, u_i = b_i + c_i + h_i$, we can see that $\rho(x_i)$ can be used to recover part of the bits of y_i, i.e., s_i and u_i.

As a result, the adversary can recover s, u using $\lceil \frac{d}{2} \rceil + 1$ probes, with the last probe placed at the LSB of q_0, the i-th probe placed the LSB of $x_i b_j + z_{ij}$ for $1 \leq i \leq \lceil \frac{d}{2} \rceil$ and $j = i + \lceil \frac{d}{2} \rceil$. Note that in this attack, we do not use of the assumptions that the adversary can probe at field-level to avoid false result that the leakage is due the stronger ability granted to the adversary. A probe placed at LSB of $x_i b_j + z_{ij}$ can recover every bit of $\rho(x_i)$ and b_j, and a probe placed at q_0 gains all the bits in $y_j + b_k$ with $0 \leq k \leq d$. Combining this information will give s_i, u_i for $1 \leq i \leq d$. Finally, with $x_0, x_0 b_0, y_0 + b_0$ given by the probe placed at q_0, the distribution of the observations must be dependent on s and u.

$$\begin{bmatrix} m \\ n \\ p \\ q \\ r \\ s \\ u \\ v \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \\ g \\ h \end{bmatrix} \tag{2}$$

B Proofs for Propositions

B.1 Proof for Proposition 2

Proof. Similar to the proof of Proposition 1, the adversary maintains a record of the observation set $\mathcal{O}$, along with its partition into $\mathcal{S}$, $\mathcal{E}$, and $\mathcal{R}$. The sets I_x and I_y are used to track the recovered domains of x and y, respectively.

Without loss of generality, we assume that the adversary places probes only at the input wires of the registers storing t_{ii} and t_{ij}, as well as at the output wires q_i, where $0 \leq i, j \leq d$ and $i \neq j$.

Initially, the adversary sets $I_x = I_y = \emptyset$. For each probe placed, they follow Process II to record both the observations and the recovered domains. We show that this process accurately tracks the observed values as well as the recovered domains of x and y. Moreover, after placing t probes, it holds that $|I_x| \leq t$ and $|I_y| \leq t$ for any $t \leq d$, which implies Proposition 2.

Base case: When $t = 0$, no probes are placed, and no domain of any sensitive variable is recovered. The claim trivially holds.

Inductive step: Assume the claim holds for $t = n$. We show it also holds for $t = n + 1$, where $0 \leq n \leq d - 1$.

Let $\mathcal{S}^t$, $\mathcal{E}^t$, and $\mathcal{R}^t$ be the partition of the observation set $\mathcal{O}$ after the adversary has placed t probes. Let $\mathcal{P}$ denote the observations from the (t+1)-th probe, and let $\mathcal{P}_s$, $\mathcal{P}_e$, and $\mathcal{P}_r$ be its partition into *shares*, *masked expressions*, and *masks*, respectively.

There are only three ways for the adversary to recover domains of sensitive variables: 1. directly from observed shares; 2. by using known masks in $\mathcal{R}$ to unmask expressions in $\mathcal{E}$; 3. by identifying multiple masked expressions in $\mathcal{E}$ that share the same mask.

The third case never yields new domain information. The only scenario where multiple expressions use the same mask is with $x_iy_j + z_{ij}$ and $x_jy_i + z_{ij}$ sharing z_{ij}. However, according to the adversary's procedure, $x_iy_j + z_{ij}$ is added to $\mathcal{O}$ only if q_i is probed, and similarly $x_jy_i + z_{ij}$ only if q_j is probed. After placing these two probes, both indices i and j are included in I_x and I_y, meaning that combining these expressions provides no further domain recovery.

Therefore, when the adversary obtains new observations $\mathcal{P}$ from the newly placed (t+1)-th probe, they only need to consider the first two recovery cases discussed previously.

First, the adversary updates the recovered domains using $\mathcal{P}_s$. Second, they check: 1. whether any mask in $\mathcal{P}_r$ can unmask a masked expression in $\mathcal{E}^t$; 2. whether any mask in $\mathcal{R}^t$ can unmask an expression in $\mathcal{P}_e$.

For probe group (1) in Process II, domain i is potentially added to I_x and I_y through $\mathcal{P}_s = \{x_i, y_i\}$. Since $\mathcal{P}_e = \emptyset$ and $\mathcal{P}_r = \emptyset$, no further consideration is needed.

For probe group (2), i and j is possibly added to I_x and I_y via $\mathcal{P}_s = \{x_i, y_i\}$, increasing $|I_x|$ or $|I_y|$ by at most one, respectively. Regarding y, we note that $z_{ij} \in \mathcal{P}_r$ might unmask $x_jy_i + z_{ij}$, provided $x_jy_i + z_{ij}$ exist in $\mathcal{O}$. At first glance, this seems to allow up to two new domains of y to be recovered. However, if i is newly added to I_y, then j must have already been included in I_y via a previous probe at q_j, which introduced $x_jy_i + z_{ij}$. Hence, at most one new domain is added to I_y in this case. Therefore, the total increase in recovered domains for x and y is still at most one, keeping the count below or equal to t+1.

For probe group (3), i may be added to both I_x and I_y via $\mathcal{P}_s$. We now analyze whether masks in $\mathcal{R}^t$ can unmask any expression in $\mathcal{P}_e$: specifically, expressions of the form $(x_iy_k + z_{ik})_{i\neq k}$.

Indeed, the random masks in K_1 (defined in Process II(3)) can be used to recover multiple new domains of y. However, there exist corresponding prior $|K_1|$ probes placed at inputs of $\{x_ky_i + z_{ik} \mid k \in K_1\}$, which previously added at most domain i or none at all to I_y. Hence, the number of y-domains recovered by the first t probes is at most $(t - |K_1| + 1)$[12]. After placing the (t+1)-th probe, up to $|K_1|$ new domains are added to I_y, giving a total of at most t+1 recovered domains.

Therefore, in all cases, the number of recovered domains for x and y is no more than t+1 after placing the (t+1)-th probe.

This completes the induction, and thus DOM-*indep* is d-NI under the glitch-extended probing model.

□

B.2 Proof for Proposition 6

Proof. Firstly, the composition of $\tilde{C}$ and a full-rank square matrix transformation is d-NI. This is because the matrix transformation only enables the adversary to obtain a glitch-extended probing set consisting of multiple bits within the same domain, similar to the case where bits in one operand of a DOM multiplier originate from different output bits of other DOM multipliers.

[12]The '+1' accounts for the current probe possibly adding i if it wasn't previously added.

Let n be the number of output bits of $\tilde{C}$, q the output bits of C, and let $\mathcal{A}$ be the full-rank square matrix transformation. Define $p = \mathcal{A}q$ as the output bits whose shares are stored in registers.

We know that q_i must be of the form $q'_i + \sum_{j \neq i} z_{ij}$, where q_i, q'_i, and z_{ij} are all in $\mathbb{F}_{2^n}$, and q'_i is the sensitive expression that needs to be masked.

After applying the matrix transformation, the output shares stored in registers become $p_i = \mathcal{A}q'_i + \sum_{j \neq i} \mathcal{A}z_{ij}$. Let $r_{ij} = \mathcal{A}z_{ij}$. Then, the set $\{r_{ij} \mid 0 \leq i, j \leq d,\ j \neq i,\ r_{ij} \in \mathbb{F}_{2^n},\ r_{ij} = r_{ji}\}$ consists of uniformly and independently distributed random elements, just like the original $\{z_{ij}\}$.

For any $i \in \{0, \ldots, d\}$ and $l \in \{1, \ldots, n\}$, the i-th share of the l-th bit of p, denoted p_i^l, is masked by d random bits $\{r_{ij}^l \mid j \neq i,\ 0 \leq j \leq d,\ r_{ij}^l \in \mathbb{F}_2\}$.

The rest of the proof mirrors that of Proposition 3.

Consider t_2 shares of a single output bit p^l. Each share p_i^l is protected by d random bits $\{r_{ij}^l \mid j \neq i,\ 0 \leq j \leq d\}$. Let $I_2 = \{i_1, \ldots, i_{t_2}\}$ be the domains of the t_2 observed shares. Then the remaining domains are $I_1 = \{0, \ldots, d\} \setminus I_2$ with $|I_1| = d + 1 - t_2$.

For any $k \in I_1$, the random bit r_{ik}^l appears only once among the t_2 observed output probes and is used solely to mask p_i^l. Thus, when considering only observations from t_2 output probes, p_i^l is protected by $d + 1 - t_2$ unique masks[13].

If the circuit were not d-MO-SNI, the adversary could use t_1 internal probes to recover r_{ik}^l for all $k \in I_1$. However, this is infeasible. The adversary cannot directly probe the l-th bit of r_{ik}; to access r_{ik}^l, they would need to recover the full z_{ik} and apply the matrix $\mathcal{A}$ to derive r_{ik}. With only t_1 internal probes, the adversary can learn at most $\{z_{ik} \mid k \in K,\ |K| \leq t_1\}$. Therefore, they can only reconstruct r_{ik} for $k \in K$. Since $|K| \leq t_1 \leq d + 1 - t_2 = |I_1|$, there exists at least one r_{ik}^l protecting p_i^l that remains unknown. Thus, the distribution of the t_2 output shares remains independent of any input shares, even in the presence of t_1 internal probes.

Finally, since $\tilde{C}$ is d-NI in the glitch-extended probing model, any t_1 internal probes can be simulated by at most t_1 shares of each input bit. Consequently, the joint distribution of the t_2 output shares and the t_1 internal probes can be simulated using only t_1 shares of each input bit for $t_1 + t_2 \leq d$.

□

References

[BBC+19] Gilles Barthe, Sonia Belaïd, Gaëtan Cassiers, Pierre-Alain Fouque, Benjamin Grégoire, and François-Xavier Standaert. maskverif: Automated verification of higher-order masking in presence of physical defaults. In Kazue Sako, Steve A. Schneider, and Peter Y. A. Ryan, editors, *Computer Security - ESORICS 2019 - 24th European Symposium on Research in Computer Security, Luxembourg, September 23-27, 2019, Proceedings, Part I*, volume 11735 of *Lecture Notes in Computer Science*, pages 300–318. Springer, 2019.

[BBD+16] Gilles Barthe, Sonia Belaïd, François Dupressoir, Pierre-Alain Fouque, Benjamin Grégoire, Pierre-Yves Strub, and Rébecca Zucchini. Strong non-interference and type-directed higher-order masking. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*, pages 116–129. ACM, 2016.

[BBP+16] Sonia Belaïd, Fabrice Benhamouda, Alain Passelègue, Emmanuel Prouff, Adrian Thillard, and Damien Vergnaud. Randomness complexity of private

[13] An additional $t_2 - 1$ masks r_{ik}^l with $k \in I_2,\ k \neq i$, are shared between p_i^l and p_k^l.

circuits for multiplication. In *Advances in Cryptology–EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II 35*, pages 616–648. Springer, 2016.

[BGN+14] Begül Bilgin, Benedikt Gierlichs, Svetla Nikova, Ventzislav Nikov, and Vincent Rijmen. Higher-order threshold implementations. In Palash Sarkar and Tetsu Iwata, editors, *Advances in Cryptology - ASIACRYPT 2014 - 20th International Conference on the Theory and Application of Cryptology and Information Security, Kaoshiung, Taiwan, R.O.C., December 7-11, 2014, Proceedings, Part II*, volume 8874 of *Lecture Notes in Computer Science*, pages 326–343. Springer, 2014.

[BGR18] Sonia Belaïd, Dahmun Goudarzi, and Matthieu Rivain. Tight private circuits: Achieving probing security with the least refreshing. In Thomas Peyrin and Steven D. Galbraith, editors, *Advances in Cryptology - ASIACRYPT 2018 - 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2-6, 2018, Proceedings, Part II*, volume 11273 of *Lecture Notes in Computer Science*, pages 343–372. Springer, 2018.

[BNN+12] Begül Bilgin, Svetla Nikova, Ventzislav Nikov, Vincent Rijmen, and Georg Stütz. Threshold implementations of all 3x3 and 4x4 s-boxes. *IACR Cryptol. ePrint Arch.*, page 300, 2012.

[BP12] Joan Boyar and René Peralta. A small depth-16 circuit for the AES s-box. In Dimitris Gritzalis, Steven Furnell, and Marianthi Theoharidou, editors, *Information Security and Privacy Research - 27th IFIP TC 11 Information Security and Privacy Conference, SEC 2012, Heraklion, Crete, Greece, June 4-6, 2012. Proceedings*, volume 376 of *IFIP Advances in Information and Communication Technology*, pages 287–298. Springer, 2012.

[Can05] David Canright. A very compact s-box for AES. In Josyula R. Rao and Berk Sunar, editors, *Cryptographic Hardware and Embedded Systems - CHES 2005, 7th International Workshop, Edinburgh, UK, August 29 - September 1, 2005, Proceedings*, volume 3659 of *Lecture Notes in Computer Science*, pages 441–455. Springer, 2005.

[CGLS21] Gaëtan Cassiers, Benjamin Grégoire, Itamar Levi, and François-Xavier Standaert. Hardware private circuits: From trivial composition to full verification. *IEEE Trans. Computers*, 70(10):1677–1690, 2021.

[CGM+24] Gaëtan Cassiers, Barbara Gigerl, Stefan Mangard, Charles Momin, and Rishub Nagpal. Compress: Generate small and fast masked pipelined circuits. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(3):500–529, 2024.

[CJRR99] Suresh Chari, Charanjit S. Jutla, Josyula R. Rao, and Pankaj Rohatgi. Towards sound approaches to counteract power-analysis attacks. In Michael J. Wiener, editor, *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 398–412. Springer, 1999.

[CMM+24] Gaëtan Cassiers, Loïc Masure, Charles Momin, Thorben Moos, Amir Moradi, and François-Xavier Standaert. Randomness generation for secure hardware masking - unrolled trivium to the rescue. *IACR Commun. Cryptol.*, 1(2):4, 2024.

[CPRR13] Jean-Sébastien Coron, Emmanuel Prouff, Matthieu Rivain, and Thomas Roche. Higher-order side channel security and mask refreshing. In Shiho Moriai, editor, *Fast Software Encryption - 20th International Workshop, FSE 2013, Singapore, March 11-13, 2013. Revised Selected Papers*, volume 8424 of *Lecture Notes in Computer Science*, pages 410–424. Springer, 2013.

[CS20] Gaëtan Cassiers and François-Xavier Standaert. Trivially and efficiently composing masked gadgets with probe isolating non-interference. *IEEE Trans. Inf. Forensics Secur.*, 15:2542–2555, 2020.

[CSV24] Gaëtan Cassiers, François-Xavier Standaert, and Corentin Verhamme. Low-latency masked gadgets robust against physical defaults with application to ascon. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(3):603–633, 2024.

[FGP+18] Sebastian Faust, Vincent Grosso, Santos Merino Del Pozo, Clara Paglialonga, and François-Xavier Standaert. Composable masking schemes in the presence of physical defaults & the robust probing model. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(3):89–120, 2018.

[G+07] VERIFICATION GROUP et al. Abc: a system for sequential synthesis and verification, release 70930, 2007.

[GKM+24] Barbara Gigerl, Franz Klug, Stefan Mangard, Florian Mendel, and Robert Primas. Smooth passage with the guards: Second-order hardware masking of the AES with low randomness and low latency. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(1):309–335, 2024.

[GMK16a] Hannes Groß, Stefan Mangard, and Thomas Korak. Domain-oriented masking: Compact masked hardware implementations with arbitrary protection order. In Begül Bilgin, Svetla Nikova, and Vincent Rijmen, editors, *Proceedings of the ACM Workshop on Theory of Implementation Security, TIS@CCS 2016 Vienna, Austria, October, 2016*, page 3. ACM, 2016.

[GMK16b] Hannes Groß, Stefan Mangard, and Thomas Korak. Domain-oriented masking: Compact masked hardware implementations with arbitrary protection order. *IACR Cryptol. ePrint Arch.*, page 486, 2016.

[GMK17] Hannes Groß, Stefan Mangard, and Thomas Korak. An efficient side-channel protected AES implementation with arbitrary protection order. In Helena Handschuh, editor, *Topics in Cryptology - CT-RSA 2017 - The Cryptographers' Track at the RSA Conference 2017, San Francisco, CA, USA, February 14-17, 2017, Proceedings*, volume 10159 of *Lecture Notes in Computer Science*, pages 95–112. Springer, 2017.

[GP99] Louis Goubin and Jacques Patarin. DES and differential power analysis (the "duplication" method). In Çetin Kaya Koç and Christof Paar, editors, *Cryptographic Hardware and Embedded Systems, First International Workshop, CHES'99, Worcester, MA, USA, August 12-13, 1999, Proceedings*, volume 1717 of *Lecture Notes in Computer Science*, pages 158–172. Springer, 1999.

[GR17] Dahmun Goudarzi and Matthieu Rivain. How fast can higher-order masking be in software? In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology - EUROCRYPT 2017 - 36th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Paris, France, April 30 - May 4, 2017, Proceedings, Part I*, volume 10210 of *Lecture Notes in Computer Science*, pages 567–597, 2017.

[HB25] Vedad Hadzic and Roderick Bloem. Efficient and composable masked AES s-box designs using optimized inverters. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2025(1):656–683, 2025.

[ISW03] Yuval Ishai, Amit Sahai, and David A. Wagner. Private circuits: Securing hardware against probing attacks. In Dan Boneh, editor, *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*, volume 2729 of *Lecture Notes in Computer Science*, pages 463–481. Springer, 2003.

[KJJ99] Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In Michael J. Wiener, editor, *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 388–397. Springer, 1999.

[KM22] David Knichel and Amir Moradi. Low-latency hardware private circuits. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, pages 1799–1812. ACM, 2022.

[KMMS22] David Knichel, Amir Moradi, Nicolai Müller, and Pascal Sasdrich. Automated generation of masked hardware. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(1):589–629, 2022.

[KNP13] Sebastian Kutzner, Phuong Ha Nguyen, and Axel Poschmann. Enabling 3-share threshold implementations for all 4-bit s-boxes. In Hyang-Sook Lee and Dong-Guk Han, editors, *Information Security and Cryptology - ICISC 2013 - 16th International Conference, Seoul, Korea, November 27-29, 2013, Revised Selected Papers*, volume 8565 of *Lecture Notes in Computer Science*, pages 91–108. Springer, 2013.

[KSM20] David Knichel, Pascal Sasdrich, and Amir Moradi. SILVER - statistical independence and leakage verification. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part I*, volume 12491 of *Lecture Notes in Computer Science*, pages 787–816. Springer, 2020.

[MCS22] Charles Momin, Gaëtan Cassiers, and François-Xavier Standaert. Handcrafting: Improving automated masking in hardware with manual optimizations. In Josep Balasch and Colin O'Flynn, editors, *Constructive Side-Channel Analysis and Secure Design - 13th International Workshop, COSADE 2022, Leuven, Belgium, April 11-12, 2022, Proceedings*, volume 13211 of *Lecture Notes in Computer Science*, pages 257–275. Springer, 2022.

[MMSS19] Thorben Moos, Amir Moradi, Tobias Schneider, and François-Xavier Standaert. Glitch-resistant masking revisited or why proofs in the robust probing model are needed. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2019(2):256–292, 2019.

[MPZ22] Maria Chiara Molteni, Jürgen Pulkus, and Vittorio Zaccaria. On robust strong-non-interferent low-latency multiplications. *IET Inf. Secur.*, 16(2):127–132, 2022.

[NRR06] Svetla Nikova, Christian Rechberger, and Vincent Rijmen. Threshold implementations against side-channel attacks and glitches. In Peng Ning, Sihan Qing, and Ninghui Li, editors, *Information and Communications Security, 8th International Conference, ICICS 2006, Raleigh, NC, USA, December 4-7, 2006, Proceedings*, volume 4307 of *Lecture Notes in Computer Science*, pages 529–545. Springer, 2006.

[RBN+15] Oscar Reparaz, Begül Bilgin, Svetla Nikova, Benedikt Gierlichs, and Ingrid Verbauwhede. Consolidating masking schemes. In Rosario Gennaro and Matthew Robshaw, editors, *Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part I*, volume 9215 of *Lecture Notes in Computer Science*, pages 764–783. Springer, 2015.

[WFP+24] Lixuan Wu, Yanhong Fan, Bart Preneel, Weijia Wang, and Meiqin Wang. Automated generation of masked nonlinear components: - from lookup tables to private circuits. In Martin Andreoni, editor, *Applied Cryptography and Network Security Workshops - ACNS 2024 Satellite Workshops, AIBlock, AIHWS, AIoTS, SCI, AAC, SiMLA, LLE, and CIMSS, Abu Dhabi, United Arab Emirates, March 5-8, 2024, Proceedings, Part I*, volume 14586 of *Lecture Notes in Computer Science*, pages 319–339. Springer, 2024.

[WGK13] Clifford Wolf, Johann Glaser, and Johannes Kepler. Yosys-a free verilog synthesis suite. In *Proceedings of the 21st Austrian Workshop on Microelectronics (Austrochip)*, volume 97, 2013.

[ZCF25] Feng Zhou, Hua Chen, and Limin Fan. Prover - toward more efficient formal verification of masking in probing model. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2025(1):552–585, 2025.